School of Computer Science
FACULTY OF ENGINEERING AND
PHYSICAL SCIENCES

Appearance-Preserving Rendering Based on HashDAG

Tianle Xie

Submitted in accordance with the requirements for the degree of
High-Performance Graphics and Games Engineering MSc

2025-2026

The candidate confirms that the following have been submitted.

Items	Format	Recipient(s) and Date
Deliverable 1	PDF Document(This Report)	Dr M Billeter, Dr H Carr(31/08/26)
Deliverable 2	Source Code Repository(`https://github.com/Sept-9/HashDAG`)	Dr M Billeter, Dr H Carr(31/08/26)

Type of project: Exploratory Software

The candidate confirms that the work submitted is their own and the appropriate credit has been given where reference has been made to the work of others.

I understand that failure to attribute material which is obtained from another source may be considered as plagiarism.

(Signature of Student) _____

Summary

High-resolution voxel scenes provide a flexible representation of complex geometry, but exact ray traversal remains expensive even when Sparse Voxel Directed Acyclic Graphs and HashDAGs substantially compress storage. This project introduces an appearance-aware level-of-detail extension that terminates traversal according to projected node size and reconstructs coarse results using multi-resolution colour, node-local directional relief, GGX roughness modulation, and coverage-aware bounded refinement. Six quantised relief values and three passability bits are stored in one inline 32-bit descriptor per geometry node. On the Epic Citadel scene, Box LOD and Relief LOD achieved mean speedups of 1.26× and 1.25×, while the complete coverage-aware method achieved 1.13× and provided the best image-quality measurements at every tested threshold. Relief improved pixel-wise and perceptual agreement over box-normal shading, and coverage-aware descent better preserved sparse structures. The descriptor increased total logical scene storage by 4.36%, although construction time increased substantially. The results demonstrate that LOD traversal and compact appearance reconstruction can be integrated into HashDAG rendering, while projected-coverage accuracy, visibility-aware relief, voxel-material filtering, and incremental maintenance under editing remain important areas for future work.

Acknowledgements

I would like to thank my supervisor, Markus Billeter, for his careful guidance and support throughout this project, which helped me complete my work.

I would also like to thank Victor Careil, Markus Billeter, and Elmar Eisemann for their HashDAG project, which provided the foundation for my project.

I would further like to express my gratitude to Microsoft Copilot and ChatGPT for their valuable assistance. Their suggestions greatly improved my efficiency and helped me overcome several technical challenges.

Finally, I would like to thank all the teachers who have taught, guided, and supported me throughout my studies.

Contents

Chapter 1

Introduction

Figure 1.1: Voxelized Sponza [4]

Voxel representations provide a regular and topology-independent way to encode complex geometry, making them particularly attractive for scenes containing fine structures, disconnected components, and geometrically intricate surfaces. The main challenge with voxel representations is scalability: the storage required by a dense voxel grid grows cubically with spatial resolution. Although sparse representations avoid allocating empty regions, real-time rendering still requires each primary and shadow ray to perform a sequence of dependent memory accesses through a large spatial hierarchy. Consequently, increasing voxel resolution affects not only memory consumption but also traversal cost and memory-bandwidth demand. Rendering high-resolution voxel scenes at interactive frame rates therefore requires both compact storage and mechanisms that avoid resolving geometric detail that cannot contribute meaningfully to the final image.

Sparse voxel octrees reduce dense-grid storage by omitting empty regions, while Sparse

Voxel Directed Acyclic Graphs further merge identical subtrees to remove repeated geometry [10, 9]. HashDAG combines DAG compression with hash-based node reuse so that affected paths can be reconstructed during editing without rebuilding the complete scene [2]. These representations substantially reduce storage, but exact rendering still descends towards the finest occupied voxels even when their detail projects to less than one pixel.

The existing hierarchy can also act as an LOD representation because each internal node bounds a progressively coarser region. Early termination can therefore reduce traversal, but treating the selected node as one opaque box loses fine occupancy, representative colour, and surface-direction variation. The resulting errors include thickened sparse geometry, block-like silhouettes, and unstable diffuse or specular shading. LOD for HashDAG must consequently approximate appearance and visibility as well as select a geometric scale.

This project integrates screen-space termination directly into HashDAG traversal and reconstructs coarse appearance using multi-resolution colour, node-local internal relief, and GGX roughness modulation. Three coverage-derived passability bits allow sparse candidate nodes to request bounded additional descent. The six relief values and passability decisions are stored in one inline 32-bit descriptor per geometry node, preserving DAG sharing without a separate runtime lookup structure. The evaluation measures the resulting trade-off between frame time, image quality, construction time, and memory consumption, while the identified visibility and editing limitations define the remaining future work.

1.1 Project Aim

The aim of this project is to develop an appearance-aware level-of-detail method for HashDAG-based voxel rendering. The method seeks to reduce ray-traversal cost while preserving the colour, visibility, and shading characteristics of fine voxel geometry.

1.2 Objectives

- Integrate screen-space LOD selection into the existing HashDAG traversal.
- Construct multi-resolution colour and node-local directional relief information.
- Use coverage-aware refinement to reduce occlusion errors in sparse geometry.
- Reconstruct coarse diffuse and specular appearance using view-dependent relief and roughness modulation.

- Evaluate rendering performance, image quality, construction time, and memory overhead.

1.3 Deliverables

- Experimental evaluation report
- Source code repository

Chapter 2

Background and Related Work

This section reviews the foundations of voxel-based scene representations and the hierarchical data structures used to store and render them efficiently. It first introduces voxel representations and sparse voxel octrees, followed by sparse voxel directed acyclic graphs and hash-based approaches for editing compressed voxel geometry. It then discusses level-of-detail rendering, appearance prefiltering, and recent approaches to appearance-preserving scene aggregation. Together, these topics establish the context for integrating level-of-detail rendering into a compressed and editable voxel representation.

2.1 Voxel-Based Scene Representations

A voxel represents a discrete region of three-dimensional space and can be regarded as the volumetric counterpart of a pixel. At its simplest, a voxel stores a binary occupancy value indicating whether the corresponding spatial region contains geometry. More general voxel representations may additionally store colour, material parameters, density, surface orientation, or other attributes required by the renderer.

Unlike polygonal meshes, which explicitly describe surfaces using vertices and connectivity, voxel representations discretise the surrounding space itself. Consequently, they do not require an explicit surface topology or parameterisation. This makes them well suited to representing geometrically complex objects, disconnected components, holes, thin structures, and spatially varying volumetric data within a uniform framework. Their regular spatial organisation also facilitates random access, constructive solid geometry operations, collision queries, and local modifications.

Early work by Meagher demonstrated how arbitrary three-dimensional objects could be represented using hierarchical octree encoding [11]. In an octree, a spatial region is recursively subdivided into eight equally sized subregions. This representation provides a natural connection between spatial scale and hierarchy, allowing coarse regions and fine geometric details to coexist within a single structure.

The principal limitation of a dense voxel representation is its poor scaling with spatial resolution. A regular grid with a linear resolution of N contains

$$N_{\text{voxels}} = N^3 \tag{2.1}$$

voxels. Doubling the resolution along each axis therefore increases the number of voxels by a factor of eight. Even when only a small amount of data is stored per voxel, a dense representation of a high-resolution scene can exceed the memory capacity of contemporary hardware. Storing additional appearance attributes further increases this cost.

Most geometric scenes occupy only a small fraction of their enclosing volume. Moreover, surface-based scenes generally contain occupied voxels primarily near a two-dimensional boundary rather than throughout the full three-dimensional domain. These observations motivate sparse hierarchical data structures that omit empty regions and preserve detail only where it is required.

2.2 Sparse Voxel Octrees

A sparse voxel octree, or SVO, recursively subdivides occupied or spatially heterogeneous regions while pruning regions that are entirely empty. The root node represents the complete scene domain, and each successive level halves the spatial extent of a node along every axis. A node may contain up to eight children, corresponding to the eight octants of its parent region.

The memory consumption of an SVO depends on the distribution of occupied regions rather than on the volume of the complete dense grid. Large empty regions can be represented implicitly without allocating nodes for every voxel they contain. This property enables SVOs to represent scenes whose nominal resolutions would be impractical for a dense grid.

The hierarchy is also useful for ray traversal. When a ray enters an empty node, the complete region represented by that node can be skipped without visiting its constituent voxels individually. Conversely, the traversal descends into finer levels only when the ray intersects a non-empty region. The octree therefore acts as both a compressed scene representation and a spatial acceleration structure.

Laine and Karras proposed an efficient SVO representation designed for GPU traversal [10]. Their method uses compact node layouts and child masks to reduce pointer and node storage overhead while supporting direct ray casting of compressed voxel geometry. Crassin et al. combined sparse voxel hierarchies with demand-driven streaming in GigaVoxels, allowing large voxel scenes to be rendered even when the complete high-resolution representation could not reside in GPU memory simultaneously [3].

Although an SVO exploits spatial sparsity, it does not exploit repeated geometric structure. Two spatially distant regions may contain identical voxel configurations, but their subtrees are stored independently in a conventional octree. Repeated architectural

elements, flat surfaces, and recurring geometric patterns may therefore produce a substantial amount of redundant node data.

The octree hierarchy also provides only a geometric notion of scale. An internal node represents the spatial extent of its descendants, but it does not necessarily provide an accurate approximation of their appearance. Treating an internal node as a solid primitive may remove holes, alter silhouettes, and discard variations in colour, surface orientation, and visibility. Consequently, the presence of an octree hierarchy alone is insufficient for appearance-preserving level-of-detail rendering.

2.3 Sparse Voxel Directed Acyclic Graphs

Sparse voxel directed acyclic graphs extend SVO compression by merging identical subtrees. Starting from the finest levels of an octree, subtrees with identical occupancy and child references are identified and represented by a single shared node. Multiple parent nodes may then refer to the same child node, transforming the tree into a directed acyclic graph.

Kämpe et al. introduced high-resolution sparse voxel DAGs as a compact representation for detailed voxelised scenes [9]. As illustrated in Figure 2.1, the reduction starts at the leaf level and progressively merges increasingly large identical subtrees. Their results demonstrated that repeated substructures can be shared while the compressed representation remains directly traversable by the renderer. This avoids the need to fully decompress the DAG into an octree before ray casting.

SVO and SVDAG compression exploit two different forms of redundancy. An SVO exploits spatial sparsity by omitting empty regions, whereas an SVDAG additionally exploits structural similarity by merging identical non-empty subtrees. The effectiveness of DAG compression therefore depends on both the geometric sparsity of the scene and the frequency with which equivalent voxel configurations occur.

Importantly, subtree merging reduces the number of unique stored nodes but does not reduce the spatial depth required to represent a given voxel resolution. A ray may still have to traverse the same number of hierarchical levels to reach the finest voxel. DAG compression should therefore primarily be understood as a reduction in storage redundancy. It may also improve cache reuse when multiple traversals access the same shared nodes, but it does not inherently shorten the geometric root-to-leaf path.

Several subsequent methods have extended the original SVDAG representation. Symmetry-aware variants seek to merge subtrees that become equivalent under spatial transformations, while other approaches address the storage of attributes in addition to binary occupancy. Dado et al. presented methods for compressing both geometry and

attributes in voxel scenes [5]. Such extensions are essential for rendering coloured or material-dependent voxel surfaces, but they also introduce additional dependencies between geometric structure and appearance data.

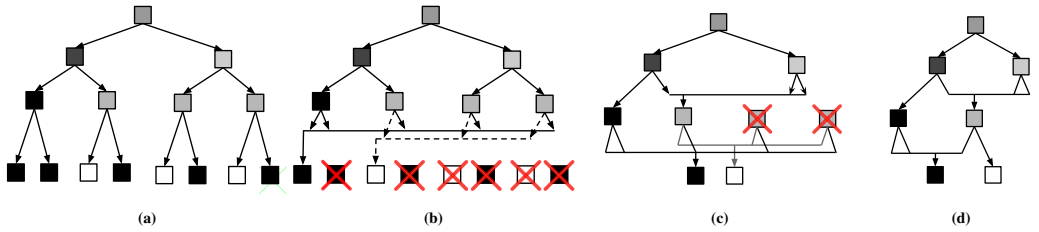

Figure 2.1: Bottom-up reduction of a sparse voxel tree into a directed acyclic graph. Identical leaves and subtrees are progressively merged and referenced by multiple parent nodes. Reproduced from Kämpe et al. [9].

2.4 Dynamic Editing of Compressed Voxel DAGs

The sharing mechanism responsible for SVDAG compression also complicates local modification. In an ordinary tree, a node has a single parent, and a local edit affects one path between the modified region and the root. In a DAG, a node may be referenced by multiple parents corresponding to different spatial regions. Modifying such a node in place would unintentionally alter every occurrence of the shared subtree.

One possible solution is to duplicate all shared nodes affected by an edit. However, repeatedly introducing independent copies would gradually remove the compression gained through subtree sharing. Alternatively, the structure could be decompressed, modified, and reconstructed after each edit, but the cost of rebuilding a large DAG is incompatible with interactive applications.

Careil et al. addressed this limitation by introducing a compressed voxel representation that supports interactive modification without requiring complete decompression and recompression [2]. The method organises nodes using hash-based deduplication. When an editing operation produces a node, its contents are used to determine whether an equivalent node is already present. Existing nodes can be reused, while genuinely new configurations are inserted into the structure. Figure 2.2 illustrates how an edit creates a new root-to-leaf path while retaining references to unaffected and already existing subtrees.

As a result, an edit reconstructs only the affected paths while preserving references to unchanged subtrees. The method supports operations such as carving, filling, copying, and painting, together with undo and redo functionality. It also supports compressed

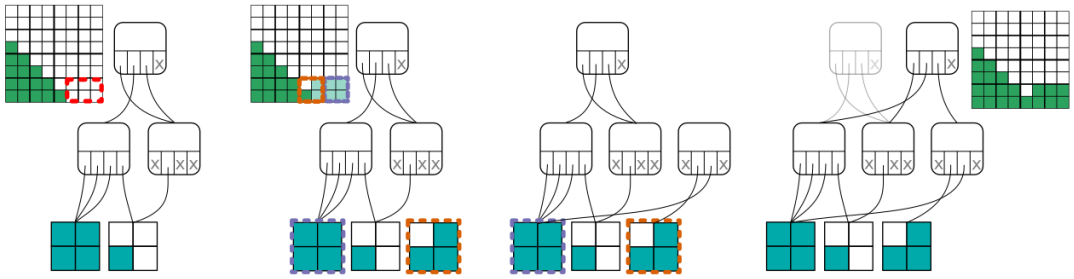

Figure 2.2: Local modification of a compressed voxel DAG. The affected leaves are replaced or reused through deduplication, after which new references are propagated towards the root. Unaffected subtrees remain shared. Reproduced from Careil et al. [2].

attributes in addition to binary geometry. This approach, commonly referred to as HashDAG, preserves the principal compression benefits of SVDAGs while making high-resolution voxel scenes interactively editable.

Nevertheless, dynamic editing and level-of-detail rendering address different aspects of the representation. Hash-based deduplication determines how modified geometry is stored efficiently, whereas level-of-detail rendering determines how much of that geometry must be traversed and evaluated for a particular view. A dynamically editable DAG therefore does not automatically provide a suitable multi-resolution appearance representation.

Furthermore, additional information attached to a shared node must be compatible with DAG deduplication. Data derived solely from the contents of a subtree can be shared together with that subtree. However, data that depends on the node's world-space position, its parent, or surrounding geometry may differ between occurrences of the same shared node. This distinction is important when designing multi-resolution attributes for compressed DAG structures.

2.5 Level-of-Detail Rendering

Level-of-detail rendering reduces computational cost by replacing geometric detail with a coarser representation when that detail has a limited contribution to the final image. The underlying assumption is that geometry projecting to a region smaller than, or comparable to, a pixel does not need to be processed at its full spatial resolution.

LOD selection may be based on camera distance, projected object size, or an estimate of screen-space error. Distance alone is inexpensive to evaluate but does not account for object size, field of view, or image resolution. A screen-space criterion provides a more direct estimate of the relationship between geometric scale and the resulting image.

Consider a hierarchical node with world-space extent s, located at an approximate

camera distance d. Let θ_p denote the angular extent represented by one pixel. Under a small-angle approximation, the projected width of the node in pixels can be estimated as

$$p \approx \frac{s}{d\theta_p}. \tag{2.2}$$

When p falls below a specified threshold, traversal may terminate at the current node rather than descending to its finest descendants. Because θ_p depends on both field of view and image resolution, this criterion is more stable under changes to camera and display configuration than a fixed world-space distance threshold.

Hierarchical voxel structures are naturally compatible with this form of LOD selection because every internal node already represents a bounded region at a particular spatial scale. Early termination can reduce node visits, memory accesses, and attribute queries. However, the visual result depends on the representation used for the terminated node.

If a sparse subtree is replaced by an opaque box, the resulting coarse representation may incorrectly fill empty space. Thin structures and partially occupied regions may become enlarged, and holes may disappear. Similarly, selecting a single descendant colour or surface normal can introduce temporal instability as the selected sample changes between frames. LOD selection must therefore be accompanied by appropriate filtering of the geometry's appearance.

This distinction can be expressed as the difference between *geometric LOD* and *appearance LOD*. Geometric LOD determines how much spatial detail is traversed or represented. Appearance LOD determines how the colour, orientation, reflectance, and visibility of the omitted detail are approximated. Reducing traversal cost without addressing appearance may improve performance while introducing objectionable aliasing and structural artefacts.

2.6 Surface Appearance Prefiltering

When multiple surfaces project to a single pixel, the pixel value is determined by the integrated response of those surfaces rather than by one arbitrarily selected sample. Without appropriate filtering, subpixel variations in texture, surface orientation, and reflectance can produce flickering, moiré patterns, unstable highlights, and incorrect energy distribution.

Texture MIP mapping addresses this problem for approximately linear colour signals by storing low-pass-filtered versions of a texture at multiple resolutions. Surface orientation and reflectance parameters are more difficult to filter because shading is a non-linear function of these quantities. In particular, averaging normal vectors does not preserve

the distribution of the original normals.

A short average normal may indicate substantial variation in the underlying orientations. Toksvig used this observation to derive an additional roughness term from the length of a filtered normal [12]. The resulting method reduces excessive specular aliasing by broadening highlights in regions containing subpixel normal variation.

Han et al. analysed normal-map filtering in the frequency domain and represented the distribution of surface orientations within a pixel footprint [6]. This shifts the interpretation from a single filtered normal to a normal distribution function. Such a distribution can preserve both the dominant orientation and the variance of the original surface, providing a more meaningful input to reflectance evaluation.

Heitz et al. proposed the SGGX microflake distribution, which represents anisotropic distributions of microscopic surface orientations using a compact symmetric matrix [7]. SGGX supports the evaluation and sampling operations required by volumetric scattering models and has become an important foundation for appearance aggregation. However, an accurate continuous distribution generally requires more storage and computation than a single colour or normal value.

These methods demonstrate that appearance preservation requires more than geometric averaging. A coarse representation should account for the distribution of orientations and its interaction with the chosen reflectance model. The appropriate representation depends on the required accuracy, the available storage, and the performance constraints of the target renderer.

2.7 Appearance Filtering in Voxel Hierarchies

A voxel hierarchy provides a sequence of spatial scales at which appearance information can be stored. Each internal node aggregates a set of finer voxels and can therefore act as a prefiltered representation of its descendants. However, meaningful aggregation requires more than the average occupancy or colour of the region.

Heitz and Neyret proposed a representation for appearance and prefiltered subpixel data in sparse voxel octrees [8]. Their method stores both macroscopic and microscopic surface descriptors within the hierarchy. The macroscopic component represents the approximate location of the surface, while the microscopic component models the distribution of unresolved surface details and attributes. Conical ray traversal selects a scale corresponding to the pixel footprint and reconstructs view- and light-dependent shading from the stored descriptors.

This work illustrates an important property of voxel LOD: selecting an octree level and filtering the appearance at that level are separate operations. The octree determines the

spatial support of a sample, whereas its stored descriptors determine how the unresolved geometry interacts with light.

Visibility is particularly difficult to aggregate. Two regions with the same occupied volume and orientation distribution can have very different transmittance if their elements are arranged differently. A continuous wall may block all rays from one direction, whereas the same amount of material distributed among disconnected elements may leave substantial gaps. Occupancy, surface orientation, and directional visibility are therefore related but not interchangeable quantities.

Furthermore, independently filtering geometry, material, and visibility may discard correlations between them. For example, two materials may be associated with surfaces that are visible from different directions. Averaging their colours independently of visibility would produce a material response that does not correspond to either view. Similar errors arise when normal orientation and occlusion are aggregated independently. These correlations are a central challenge in appearance-preserving LOD rendering.

2.8 Appearance-Preserving Scene Aggregation

Recent work has increasingly formulated LOD generation as a problem of scene appearance aggregation rather than geometric simplification. The objective is to construct a representation whose rendered images remain close to those of the original scene across viewing distances, illumination conditions, and material configurations.

Bako et al. introduced Deep Appearance Prefiltering, a neural framework for representing complex geometry and material appearance across multiple scales [1]. Rather than relying on a fixed analytic parameterisation, the method learns a compact latent representation of the filtered scene appearance. This improves its ability to model complex combinations of geometry and reflectance, but requires a scene-specific training process and neural-network evaluation during rendering.

Weier et al. proposed a neural prefiltering method designed to retain visibility correlations across levels of detail [14]. Their work addresses artefacts such as light leaking that occur when visibility is represented only by independent local averages. By learning appearance and visibility components, the method preserves information that is difficult to capture using a small set of conventional volumetric parameters.

Zhou et al. introduced Appearance-Preserving Scene Aggregation and defined the Aggregated Bidirectional Scattering Distribution Function, or ABSDF [16]. The ABSDF describes the aggregate appearance of the surfaces contained within a spatial region. It accounts for material response, surface orientation, and visibility, thereby extending the concept of a conventional surface scattering function to aggregated scene geometry.

Their formulation uses an SGGX distribution to represent surface orientation statistics. Truncated ellipsoids are used as aggregation primitives to better preserve local correlations between geometry and visibility than a simple cubic approximation. The method also records aggregated interior and boundary visibility to account for longer-range correlations. These components allow the representation to retain view-dependent and high-frequency effects while providing practical evaluation and sampling procedures.

Appearance aggregation methods offer substantially greater fidelity than replacing detailed geometry with a single averaged surface. However, this accuracy is accompanied by additional storage, precomputation, and rendering cost. Analytic volumetric models require multiple parameters to describe orientation and visibility, while neural methods require training and network inference. The appropriate balance therefore depends on whether the primary objective is physical accuracy, compact storage, real-time performance, dynamic modification, or some combination of these requirements.

2.9 Research Gap

Existing research provides effective solutions to several individual aspects of high-resolution voxel rendering. SVOs exploit spatial sparsity, while SVDAGs additionally exploit repeated geometric structure. Hash-based DAG representations support local modifications without reconstructing the complete compressed scene. Separately, surface and volumetric prefiltering methods demonstrate how unresolved geometry can be represented using filtered material, orientation, and visibility information.

Nevertheless, efficiently combining these properties remains challenging. A conventional HashDAG is designed primarily around exact occupancy and subtree sharing rather than view-dependent multi-resolution appearance. Conversely, comprehensive appearance-aggregation models are generally designed for static, prefiltered assets and may require considerably more data per spatial region than a compact DAG node.

A practical multi-resolution DAG representation must satisfy several competing requirements. First, LOD selection should be related to screen-space contribution so that the chosen resolution remains meaningful under changes in viewing distance, field of view, and image resolution. Second, the representation of a coarse node should preserve the most visually significant properties of its descendants, including colour, surface orientation, and directional visibility. Third, the additional information should not eliminate the storage benefits obtained through subtree sharing. Finally, the rendering cost of accessing and evaluating the filtered representation must remain lower than the traversal work it replaces.

These requirements reveal a gap between geometry-only LOD termination and comprehensive appearance aggregation. Geometry-only approaches are compact and inexpensive but may introduce silhouette, shading, and visibility errors. More complete analytic or neural representations can achieve higher fidelity, but their storage, preprocessing, or evaluation cost may be unsuitable for a real-time compressed voxel renderer.

This motivates the investigation of lightweight appearance-aware LOD representations for hash-compressed voxel DAGs. Such representations seek to preserve the principal visual effects of unresolved geometry while retaining the compactness and traversal efficiency that make SVDAG and HashDAG structures attractive for high-resolution voxel scenes.

Chapter 3

Method

3.1 Method Overview

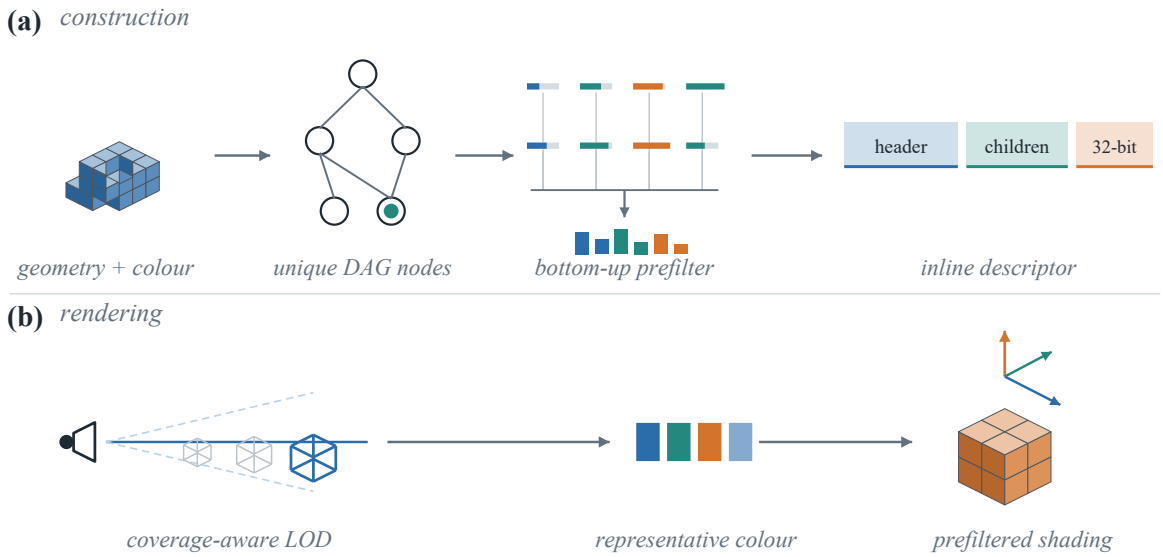

Figure 3.1: Overview of the proposed HashDAG LOD pipeline. During construction, geometry and colour are organised into unique DAG nodes, and a bottom-up prefilter produces an inline descriptor for each node. During rendering, coverage-aware LOD selects a termination node, after which its representative colour and prefiltered directional information are used for shading.

This chapter presents an appearance-aware level-of-detail (LOD) rendering method for high-resolution voxel scenes represented by a HashDAG. The method reduces traversal cost by allowing a primary ray to terminate at an internal DAG node when further descent would resolve less detail than required by the current pixel footprint. Early termination alone, however, replaces a potentially sparse and geometrically complex subtree with a single axis-aligned box. This substitution can obscure background geometry and discard the colour and surface-direction variation required for stable shading. The proposed method therefore couples screen-space termination with node-local appearance and directional-coverage information.

Figure 3.1 summarises the construction and rendering stages. During offline construction, geometry and compressed colour are organised into the unique nodes of the HashDAG. A bottom-up prefilter then reduces the geometry below each node to six

directional relief values and three coverage-derived passability indicators, which are quantised into one inline 32-bit descriptor. Because this descriptor is derived from the node's subtree, all spatial instances that reference the same deduplicated node also share the same prefiltered result.

At runtime, hierarchical ray traversal initially follows the original HashDAG path. Each visited node is tested against a screen-space LOD criterion based on its projected size. The inline descriptor is consulted only after a node becomes eligible for early termination. A coverage-derived passability bit associated with the dominant ray axis may request a bounded amount of additional descent, reducing premature occlusion in sparse regions. Once traversal terminates, the selected node supplies a representative colour and a node-specific internal-relief distribution. The geometric normal of the intersected box face is retained for the resolved boundary, while the relief distribution accounts for unresolved internal surfaces and modulates the diffuse and GGX responses.

The resulting design separates three related decisions. Projected size determines whether a node is sufficiently small to become an LOD candidate; directional coverage determines whether that candidate should be refined further; and the prefiltered appearance model determines how the final coarse hit is shaded. This separation permits each component to be evaluated independently while keeping the runtime representation compact and local to the traversed DAG node.

3.2 Baseline HashDAG Representation and Rendering

This section defines the geometry and rendering conventions required by the proposed LOD extension. The general compression principles of SVDAGs and HashDAGs were discussed in Chapter 2; here, the description is restricted to the node layout, coordinate representation, and rendering stages that are directly modified or consumed by the method.

3.2.1 Geometry Representation

The scene occupies a cubic voxel domain recursively subdivided into eight octants. The root is assigned level $l = 0$, and L denotes the full spatial depth at which one grid cell corresponds to one finest-level voxel. A node at level l consequently covers an axis-aligned region with edge length

$$S_l = 2^{L-l} \tag{3.1}$$

in finest-voxel units. Descending by one level selects one of eight children and halves the represented edge length along all three axes.

Ignoring the auxiliary descriptor introduced later in this chapter, an internal HashDAG node consists of a header followed by references to its existing children. The lowest eight bits of the header form a child mask, so empty octants require neither a child reference nor a stored node. References are packed in child order, and the location of a particular reference is obtained by counting the set mask bits that precede it. The remaining header fields include the number of enclosed surface voxels required by the compressed colour representation.

The last two binary subdivision levels are represented differently. A leaf at level $L - 2$ covers a $4 \times 4 \times 4$ region and stores its 64 binary occupancy values directly in two 32-bit words. The traversal can derive the child masks for the final two descent steps from this 64-bit value without allocating additional interior nodes. Thus, the representation combines variable-length internal nodes with fixed-size occupancy leaves.

A spatial path is represented by three integer coordinates. At each descent, the three bits of the selected child index are appended to the corresponding x, y, and z coordinates. At full depth, these coordinates identify the finest voxel reached by the ray. HashDAG compression does not change this spatial interpretation: several parent paths may reference the same stored node, but each traversal retains its own path coordinates. Geometry and colour are stored in associated but distinct compressed structures, allowing identical geometry to remain shared even when its spatial instances require different colour data.

3.2.2 Baseline Rendering Pipeline

The renderer separates primary-path traversal, colour retrieval, and illumination into successive GPU stages. In the baseline configuration, primary traversal begins at the root and continues until it reaches an occupied finest-level voxel. This is a hierarchical ray traversal rather than a conventional DDA over a dense regular grid. Each stack entry records a node reference, its child mask, and the subset of children intersected by the ray. Occupied children are visited in an order determined by the signs of the ray direction. When no candidate child remains, traversal ascends to the nearest stack entry with an unvisited intersection.

For an internal node, the child mask first removes empty octants. A ray–box slab test then determines which remaining children can be intersected and supplies the entry and exit parameters used by the traversal. Near the bottom of the hierarchy, the same intersection logic is applied to child masks decoded from the 64-bit leaf occupancy. The baseline termination condition is reached only when the traversal level equals L. The resulting full-resolution path is written to an intermediate path buffer for the following stages.

The colour stage follows the stored spatial path through the geometry and colour structures to locate the colour associated with the intersected surface voxel. The illumination stage reconstructs the exact hit point on the unit voxel box, obtains the corresponding axis-aligned face normal, evaluates shadow visibility, and applies the configured diffuse and specular BRDF together with fog. Consequently, the baseline pipeline obtains colour and shading information only after resolving a finest-level voxel. The remaining sections extend this pipeline so that a path may instead terminate at an internal node while still providing a stable colour, an appropriate boundary normal, and a compact approximation of the unresolved directional surface distribution.

3.3 Screen-Space LOD Selection

The baseline traversal resolves every visible surface at the finest voxel level, even when a node and all of its descendants project to a fraction of a pixel. The proposed method instead derives a per-node screen-space criterion that identifies when further descent is unlikely to reveal resolvable spatial detail. The criterion accounts for the camera projection, output resolution, node scale, and ray-entry distance rather than relying on a fixed world-space distance threshold.

3.3.1 Pixel Footprint Estimation

The renderer constructs primary rays from a planar virtual screen. Let $\Delta\mathbf{y}$ denote the world-space displacement between vertically adjacent pixel samples on that screen, let $\mathbf{o}$ be the camera position, and let $\mathbf{x}_c$ be the screen centre. The angular extent of one pixel is approximated by

$$\theta_p \approx \frac{\|\Delta\mathbf{y}\|}{\|\mathbf{x}_c - \mathbf{o}\|}. \tag{3.2}$$

This is the small-angle form of the angle subtended by one vertical pixel. It is evaluated once from the current camera and image-plane configuration and is therefore shared by all primary rays in the frame. A wider field of view or a lower vertical resolution increases θ_p, whereas a narrower field of view or a higher resolution decreases it. The termination test consequently adapts to changes in both camera projection and output resolution.

3.3.2 Projected Node Size

Consider a ray entering a node at level l. Its edge length S_l is given by Equation 3.1. Let d denote the distance from the camera to the ray-entry point of the node. The ray direction is normalised, so d is obtained directly from the entry parameter of the node's

slab intersection. Using the entry point rather than the node centre avoids treating the node as farther away than its nearest visible extent.

At distance d, the world-space width represented by one pixel is approximately $d\theta_p$. The projected width of the node, measured in pixels, is therefore estimated as

$$p_l \approx \frac{S_l}{d\theta_p}. \tag{3.3}$$

This expression is a conservative scalar approximation for an axis-aligned cubic node. It does not attempt to calculate the exact projected polygon of the box; instead, it provides a low-cost scale estimate that can be evaluated within the existing hierarchical traversal. Since S_l halves after every descent, p_l decreases predictably as the ray moves to finer levels.

3.3.3 Early-Termination Criterion

Let τ be the user-controlled LOD threshold in pixels. A node becomes eligible for early termination when $p_l < \tau$. Substituting Equation 3.3 gives the equivalent runtime comparison $S_l < d\theta_p\tau$.

The product $\theta_p\tau$ depends only on the camera, output resolution, and chosen threshold, so it is precomputed once and passed to the traversal as an LOD scale. The node entry distance and edge length are already available when the child intersection is evaluated. The projected-size test therefore reuses the existing slab result instead of performing a second ray–box intersection.

A larger value of τ makes coarser nodes eligible and favours traversal performance, while a smaller value postpones termination and preserves more geometric detail. Setting $\tau = 0$ disables LOD and restores full descent to level L. Passing the projected-size test identifies only an LOD candidate: the coverage-derived passability test described later in this chapter may require additional descent before the final termination level is accepted.

3.4 Multi-Resolution Colour Representation

Early termination selects an internal geometry node rather than a single finest-level voxel. The original per-voxel colour lookup is therefore no longer sufficient: the selected region may contain many differently coloured surface voxels, and choosing one of them would introduce unstable spatial and temporal variation. The associated colour hierarchy is extended with deterministic representative colours that can be retrieved at the same spatial scale as the terminated geometry path. This representation remains

separate from the inline geometry descriptor introduced in Section 3.8, because geometry deduplication and colour compression have different sharing domains.

3.4.1 Internal-Node Colour Aggregation

Let $\widetilde{\mathbf{c}}_i$ be the representative RGB colour of child i, and let w_i be the number of occupied surface voxels represented by that child. The representative colour of an internal colour node n is estimated recursively as

$$\overline{\mathbf{c}}_n = \phi^{-1}\left(\frac{\sum_{i\in\mathcal{C}(n)} w_i\,\phi(\widetilde{\mathbf{c}}_i)}{\sum_{i\in\mathcal{C}(n)} w_i}\right), \tag{3.4}$$

where $\mathcal{C}(n)$ contains the non-empty children and ϕ denotes the configured conversion into the averaging space. Weighting by surface-voxel count prevents a small child region from contributing as strongly as a densely occupied one. In the current configuration, ϕ is the identity transformation and colours are averaged in their stored RGB space. The implementation can instead select linear-RGB averaging without changing the hierarchy or runtime lookup. This construction-time choice is independent of the later conversion used by the illumination model.

At the base of the compressed colour hierarchy, a stored colour range may itself represent several surface voxels. Its initial representative is estimated from at most eight deterministic samples distributed around the centre of the range. These base estimates are then propagated bottom-up using Equation 3.4. The resulting representative is quantised to RGB888 and stored alongside the corresponding internal colour node. Because the base values are sampled rather than exhaustively decoded, the hierarchy provides a bounded-cost approximation rather than an exact arithmetic mean of every enclosed colour.

3.4.2 Colour Retrieval at Different Termination Levels

Colour retrieval depends on the level at which geometry traversal terminates. If the selected geometry node lies strictly above the colour-leaf region, the renderer follows the spatial path to the corresponding internal colour node and reads its precomputed RGB888 representative directly. The lookup cost is therefore proportional to the accepted path depth and does not require visiting the terminated node's descendants.

If termination occurs within the compressed colour-leaf region, the path identifies a contiguous colour range rather than a stored internal average. The renderer draws up to K deterministic samples from that range and averages them in the same configured colour space. For a range containing N colours, sample s is selected near the centre of its equal-width interval, with the implementation clamping the resulting index to the

valid range. The evaluated renderer uses $K = 1$, which selects the central representative of the identified colour range. This deterministic lookup avoids sampling noise between frames and bounds the cost of colour retrieval, while coarser nodes above the colour-leaf region continue to use their recursively precomputed averages. When LOD is disabled, the original full-resolution path and exact per-voxel colour lookup are retained. Thus, colour approximation is introduced only for rays that actually accept an internal termination region.

3.5 Node-Local Directional Surface Representation

Replacing a subtree by its enclosing box removes the orientations of all surfaces below the selected level. A scalar occupancy or coverage value cannot reconstruct this directional information. The method therefore represents each node by axis-aligned surface statistics derived from its own subtree. These statistics are computed independently at every stored hierarchy level: two nodes at the same level may have different distributions, and the same spatial region has a different description when aggregated at a different level.

3.5.1 Six-Direction Surface Representation

Let

$$\mathcal{D} = \{-X, +X, -Y, +Y, -Z, +Z\} \tag{3.5}$$

denote the six axis directions, with corresponding unit normals $\mathbf{n}_d$. For a node N, let Ω_N be the set of occupied finest-level voxels in its cubic support. The exterior of this support is treated as empty while the node-local statistics are constructed. The directional exposed area is

$$A_d(N) = \sum_{\mathbf{x} \in \Omega_N} \mathbb{I}[\mathbf{x} + \mathbf{e}_d \notin \Omega_N], \qquad d \in \mathcal{D}, \tag{3.6}$$

where $\mathbf{e}_d$ is the unit grid displacement in direction d, and each exposed finest-voxel face contributes one unit of area. Consequently, A_d counts both faces on the outer boundary of the node and exposed faces surrounding cavities or disconnected components inside it. Unlike the projected area of a solid box, A_d can exceed S_l^2 when several surface layers with the same orientation occur within the node.

The occupied area on the corresponding boundary slab is recorded separately as

$$B_d(N) = \sum_{\mathbf{x} \in \Omega_N} \mathbb{I}[\mathbf{x} \in \partial_d N] \,, \tag{3.7}$$

where $\partial_d N$ is the one-voxel-thick face of the node with outward normal $\mathbf{n}_d$. Hence $0 \leq B_d \leq S_l^2$. Retaining A_d and B_d separately is essential: A_d contains the directional surface information required by both coverage selection and appearance reconstruction, whereas B_d identifies the portion already represented by the boundary of the coarse ray–box hit.

The six bins form an axis-aligned area distribution rather than an estimated single normal. This representation is compatible with voxel geometry, inexpensive to aggregate, and invariant under the number of spatial references to a deduplicated node. View dependence is introduced only during shading; the stored values themselves depend solely on the occupancy represented by the particular node.

3.5.2 Boundary and Internal-Relief Separation

Directly normalising the exposed areas produces the raw directional histogram

$$h_d^{\text{raw}}(N) = \frac{A_d(N)}{S_l^2}. \tag{3.8}$$

Although this histogram describes directional area, it is not suitable for direct coarse shading because it does not distinguish the external boundary intersected by the ray from unresolved structure inside the node. The ray–box intersection already supplies a unique geometric normal for the selected boundary face. Including the node boundary again in the histogram both duplicates this resolved surface and introduces faces that the ray did not hit.

A completely solid node makes the problem explicit. For every direction,

$$A_d(N) = B_d(N) = S_l^2 \quad \Longrightarrow \quad h_d^{\text{raw}}(N) = 1. \tag{3.9}$$

For an oblique view, a front-facing test of the form

$$\max(0, \mathbf{n}_d \cdot \mathbf{V}) \tag{3.10}$$

can retain up to three of these six axis normals. Shading with the raw histogram would therefore blend the normal of the intersected face with two unrelated faces of the solid node. This is unnecessary and incorrect: at the selected scale, a solid node is exactly an

axis-aligned box, and its actual ray-entry face is already known.

The proposed representation removes the boundary contribution before normalisation. The internal-relief value for direction d is

$$r_d(N) = \text{clamp}\left(\frac{A_d(N) - B_d(N)}{S_l^2}, 0, 1\right). \tag{3.11}$$

The subtraction reserves the geometric box normal for the resolved hull while retaining directional surfaces that are hidden by aggregation. Clamping is required because repeated internal layers can make $A_d - B_d > S_l^2$, whereas the compact descriptor stores a normalised directional weight rather than an unbounded surface count. For the solid node in Equation 3.9, all six relief values are zero. Its shading therefore falls back entirely to the correct normal of the intersected box face. Internal surfaces contribute only when $A_d > B_d$.

Figure 3.2 illustrates this separation for four 4^3 nodes at one common LOD level. A solid block has occupied area on all six boundary slabs but no internal relief. A slab placed inside the node retains the two directions perpendicular to the slab; a staircase retains its unresolved top and riser surfaces; and a porous node produces a broader directional distribution. These are examples of separate node descriptors at the same aggregation scale, not globally prescribed values for the level.

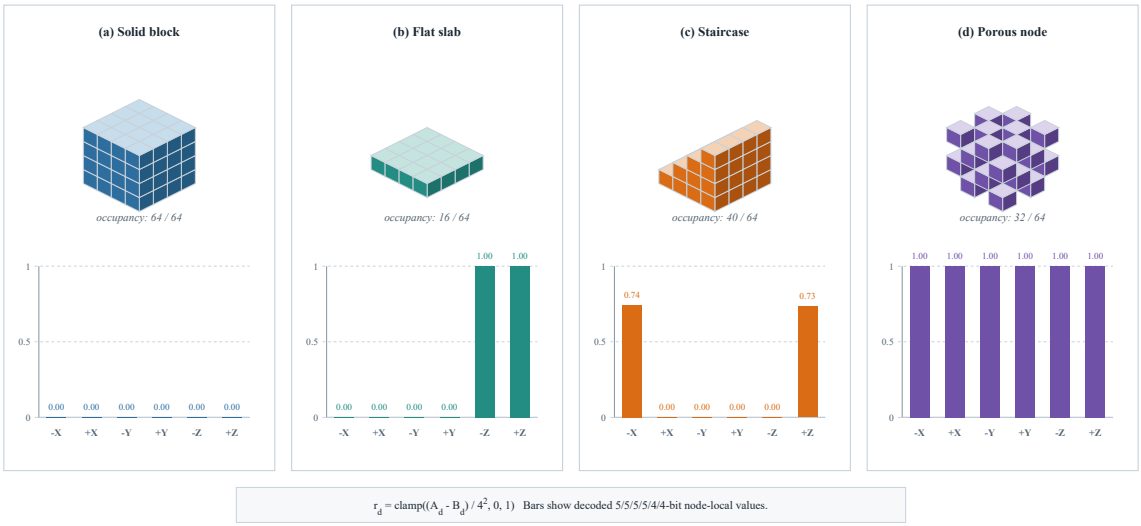

Figure 3.2: Boundary and internal-relief separation for four 4^3 nodes at the same LOD level. The upper row shows occupied finest-level voxels; the lower row shows the corresponding six-direction internal-relief values. Subtracting the occupied boundary slabs makes the solid block produce zero relief, so a coarse ray hit uses only its correct box-face normal. The internal slab, staircase, and porous node retain directional area that is not represented by the node hull.

3.5.3 Exact Leaf-Level Computation

The 4^3 geometry leaf provides an exact base case for the directional statistics. Its occupancy is stored in a 64-bit mask M, with each bit corresponding to one finest-level voxel. Rather than iterating over the 64 cells and testing six neighbours independently, the construction translates the complete mask by one voxel along each axis and uses bitwise difference tests.

Let $T_{+a}(M)$ and $T_{-a}(M)$ denote one-cell translations of M along the positive and negative directions of axis a. Bits shifted outside the 4^3 support are discarded, implementing the node-local convention that the exterior is empty. The exposed areas are then obtained exactly as

$$A_{-a} = \text{popcount}(M \,\&\, \neg T_{+a}(M)) \,, \tag{3.12}$$

$$A_{+a} = \text{popcount}(M \,\&\, \neg T_{-a}(M)) \,. \tag{3.13}$$

For example, an occupied bit remains in the first expression only when the adjacent voxel in direction $-a$ is empty. The same operations are applied to X, Y, and Z, yielding all six values without decoding an explicit voxel array.

Fixed masks F_d select the 16 bit positions on each outer face of the leaf. The boundary areas are therefore

$$B_d = \text{popcount}(M \,\&\, F_d), \qquad d \in \mathcal{D}. \tag{3.14}$$

Equations 3.12–3.14 compute integer face counts directly from the stored occupancy, so the leaf-level result introduces no geometric approximation. Approximation is required only when these values are combined across deduplicated internal nodes, as described in the following section.

3.6 Hierarchical Prefilter Construction

The exact leaf statistics are propagated towards the root after DAG construction. Each recursive result contains twelve floating-point accumulators: exposed area A_d and boundary area B_d for the six directions. Only the final normalised values are quantised; retaining unquantised accumulators during construction avoids compounding quantisation error across hierarchy levels.

3.6.1 Bottom-Up Aggregation

Let $\mathcal{C}(N)$ be the non-empty children of an internal node N. Before accounting for contacts between siblings, the directional exposed area is the sum of the child values,

$$A_d^{\Sigma}(N) = \sum_{C_i \in \mathcal{C}(N)} A_d(C_i). \tag{3.15}$$

The boundary of the parent receives contributions only from children that touch the corresponding parent face. If $\mathcal{H}_d(N) \subseteq \mathcal{C}(N)$ is this set of at most four children, then

$$B_d(N) = \sum_{C_i \in \mathcal{H}_d(N)} B_d(C_i). \tag{3.16}$$

This boundary aggregation is exact. The four child faces tile the parent face without overlap, and empty child octants contribute zero. The exposed-area sum in Equation 3.15, by contrast, temporarily treats each child as an isolated region. Faces on a shared sibling interface are counted even when the adjacent sibling occupies the same location and hides them.

3.6.2 Sibling-Interface Correction

Consider a pair of children (C_i, C_j) adjacent along axis a, with C_i on the low side and C_j on the high side. Their touching slabs have occupancies $B_{+a}(C_i)$ and $B_{-a}(C_j)$. If a full two-dimensional occupancy mask were retained for every slab, the covered interface could be evaluated by an exact mask intersection. Storing those masks at all levels would substantially increase construction state and undermine the compact node-local representation. The method instead estimates the occupied intersection from the two scalar areas:

$$\widehat{I}_{ij}^{a} = \frac{B_{+a}(C_i) B_{-a}(C_j)}{S_c^2}, \tag{3.17}$$

where $S_c = S_l/2$ is the child edge length and S_c^2 is the area of one touching slab. Equation 3.17 assumes that occupied positions are independently distributed across the two slabs. It is exact when either slab is empty or completely occupied; otherwise, it is a bounded-cost approximation because the scalar counts do not encode spatial correlation.

Summing over the four possible low–high child pairs perpendicular to axis a gives

$$\widehat{I}_a(N) = \sum_{(i,j)\in\mathcal{I}_a(N)} \widehat{I}^a_{ij}. \tag{3.18}$$

The same physical overlap hides a positive-facing surface of the low child and a negative-facing surface of the high child. It is therefore subtracted once from each corresponding directional sum:

$$A_{+a}(N) = \max\left(0, A^{\Sigma}_{+a}(N) - \widehat{I}_a(N)\right), \tag{3.19}$$

$$A_{-a}(N) = \max\left(0, A^{\Sigma}_{-a}(N) - \widehat{I}_a(N)\right). \tag{3.20}$$

The non-negativity clamp protects against accumulated floating-point approximation error. Applying the correction at every internal node prevents sibling contacts from being repeatedly interpreted as exposed relief as the hierarchy is reduced. Figure 3.3 summarises the exact leaf computation, bottom-up aggregation, interface correction, and final descriptor generation.

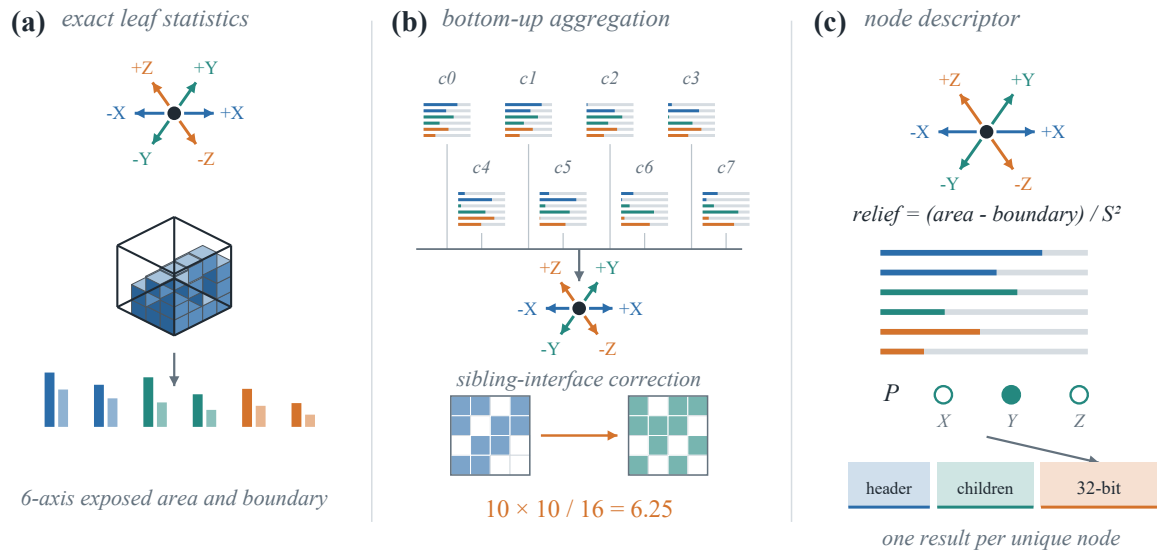

Figure 3.3: Bottom-up construction of the node-local prefilter. (a) Exact leaf processing measures exposed and boundary area along six axis directions. (b) The eight child descriptors are aggregated and corrected for hidden sibling interfaces. (c) The corrected areas produce node-local relief and coverage-derived passability, which are stored as one inline 32-bit descriptor per unique DAG node.

3.6.3 DAG-Aware Memoisation

A direct recursive traversal of the logical octree would recompute a shared subtree once for every incoming reference, discarding the main computational benefit of DAG

compression. Prefilter construction is therefore memoised by the stored HashDAG node index. These indices are globally unique across hierarchy levels in the implementation, so the index alone identifies both the node contents and their aggregation scale.

When construction first reaches a node, it recursively obtains the statistics of its existing children, applies Equations 3.15–3.20, emits the node's packed descriptor, and inserts the unquantised (A_d, B_d) result into a temporary cache. Every later reference to the same node returns that cached result without descending again. Thus, a unique node is evaluated once irrespective of how many spatial instances reference it.

This reuse is valid because all quantities are functions only of the stored subtree and its node level. They do not depend on the parent path, camera, light, or a particular DAG instance. The temporary construction cache is released after the inline descriptors have been produced; it is not required by traversal or shading at runtime.

3.7 Coverage-Derived Directional Passability

The projected-size criterion treats every occupied node as an opaque coarse box. This approximation is effective for dense regions but can cause thin branches, foliage, fences, and other sparse structures to occlude geometry that remains visible through their descendants. To identify such cases without tracing additional rays, the method derives one passability decision for each principal axis from the same exposed-area statistics used by the appearance prefilter.

For axis $a \in \{X, Y, Z\}$, the bidirectional coverage measure is

$$C_a(N) = \frac{A_{-a}(N) + A_{+a}(N)}{2S_l^2}. \tag{3.21}$$

Averaging the two opposing directions makes the measure independent of the sign from which the node is approached. A solid node gives $C_a = 1$ for every axis, whereas a sparsely occupied node generally produces a smaller projected directional area. Repeated internal layers may yield $C_a > 1$; no clamp is required because passability depends only on comparison with a lower threshold.

The stored axis decision is

$$P_a(N) = \mathbb{I}[C_a(N) < T_{\text{cov}}]\,. \tag{3.22}$$

The current configuration uses the compile-time value $T_{\text{cov}} = 0.35$. Therefore, $P_a = 1$ marks a low-coverage node through which a ray dominated by axis a may plausibly

pass, while $P_a = 0$ marks a node sufficiently covered to accept coarse termination. The three stored bits are thresholded passability predicates, not a three-bit quantisation of coverage; changing the configured threshold changes their meaning directly.

Coverage and internal relief use related source statistics but serve different decisions. Equation 3.21 uses the complete exposed areas A_d, including boundary contributions, because traversal needs an estimate of whether the node occupies the ray's projected direction. Equation 3.11 subtracts B_d, because shading must avoid counting the resolved box boundary twice. A node can consequently have zero internal relief yet remain non-passable, as occurs for a solid block. Conversely, a sparse node can contain substantial internal directional structure and still request further descent.

At runtime, the ray direction selects one of the three predicates rather than a signed face value. The precise dominant-axis rule and bounded refinement policy are described in Section 3.10. Figure 3.4 previews this interaction between screen-space eligibility and coverage-aware refinement.

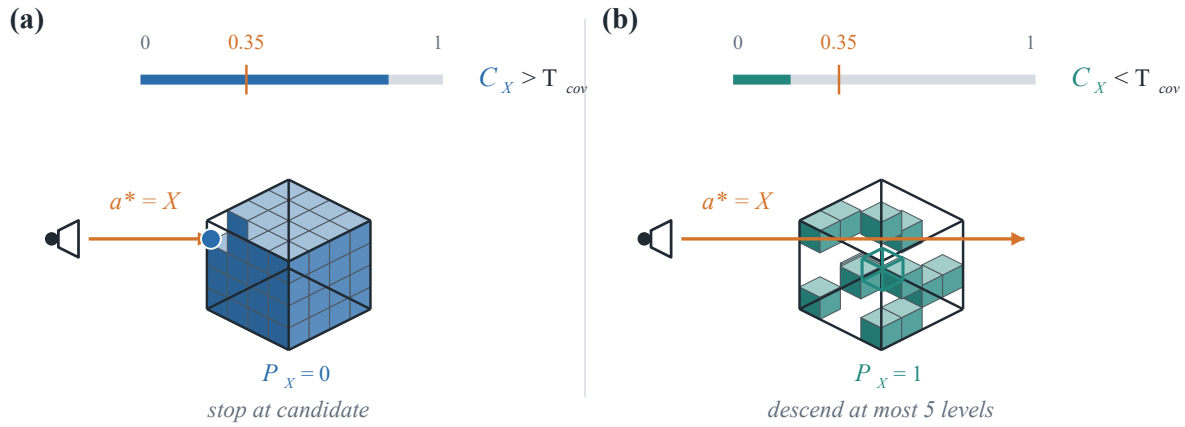

Figure 3.4: Coverage-aware refinement after screen-space LOD selection. For the dominant ray axis X, a well-covered node with $C_X > T_{\text{cov}}$ stores $P_X = 0$ and terminates at the candidate. A sparse node with $C_X < T_{\text{cov}}$ stores $P_X = 1$ and descends into a child, subject to the maximum budget of five additional levels.

3.8 Compact Inline 32-Bit Descriptor

The six floating-point relief values and three Boolean passability decisions are reduced to one unsigned 32-bit word per stored geometry node. For a relief field containing b_d bits, the encoder computes

$$q_d = \operatorname{round}\left[r_d \left(2^{b_d} - 1\right)\right], \tag{3.23}$$

and the decoder reconstructs $\widehat{r}_d = q_d/(2^{b_d} - 1)$. Since r_d has already been clamped to

$[0, 1]$, the endpoint values are represented exactly. Quantisation occurs only after the unquantised hierarchy construction is complete.

The fields follow the direction order $(-X, +X, -Y, +Y, -Z, +Z)$. The four X- and Y-direction bins use five bits each, while the two Z-direction bins use four bits each, giving the 555544 allocation. Specifically, bits 0–4 encode $-X$, bits 5–9 encode $+X$, bits 10–14 encode $-Y$, bits 15–19 encode $+Y$, bits 20–23 encode $-Z$, and bits 24–27 encode $+Z$. Bits 28, 29, and 30 store P_X, P_Y, and P_Z, respectively; bit 31 is reserved and remains unused in the current method. The complete budget is

$$4 \times 5 + 2 \times 4 + 3 + 1 = 32 \text{ bits}. \tag{3.24}$$

The asymmetric relief allocation preserves the original 28-bit directional format while recovering the three high bits for coverage-aware traversal and retaining one reserved bit. A packed value with all six relief fields equal to zero naturally selects pure box-normal shading; no additional validity flag is needed. Passability is decoded independently from bits 28–30, so a zero relief distribution does not imply any particular coverage decision.

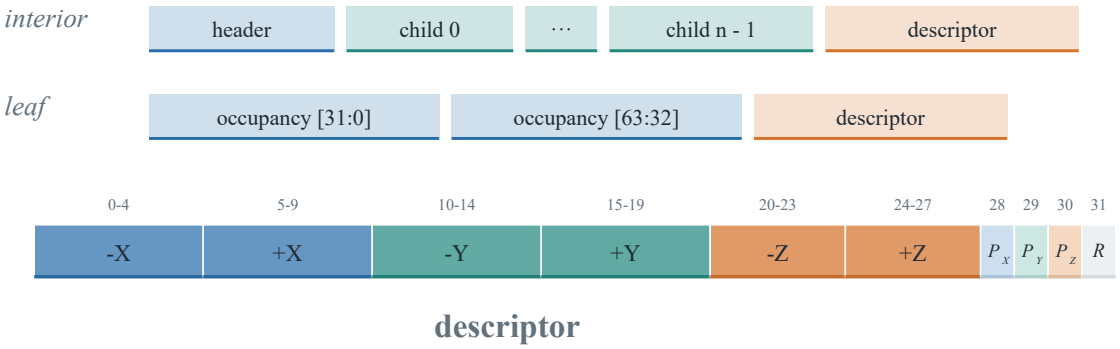

Figure 3.5: Inline descriptor and node-memory layout. Six quantised internal-relief values occupy bits 0–27 using the 555544 allocation, the coverage-derived passability flags P_X, P_Y, and P_Z occupy bits 28–30, and bit 31 is reserved. The descriptor is appended to both interior and leaf nodes.

3.9 Integration with the HashDAG

The descriptor is stored inline at the end of every allocated geometry node. An interior node has the physical layout

$$[\,\text{header}\,][\,\text{existing child references}\,][\,\text{prefilter}\,], \tag{3.25}$$

where the number of references is the population count of the header's child mask. A 4^3

leaf has the fixed layout

$$[\,\text{occupancy}_{0\ldots31}\,][\,\text{occupancy}_{32\ldots63}\,][\,\text{prefilter}\,]. \tag{3.26}$$

Consequently, a descriptor read requires only the node address and its existing logical size. Interior access computes an offset of one plus the number of child references; leaf access uses the fixed offset of two words. No secondary pointer, sparse lookup table, or per-instance record is required. Node iteration and bucket allocation use the physical size including the trailing word, ensuring that variable-length nodes remain correctly aligned in the HashDAG memory pool.

The descriptor is deliberately excluded from node identity. Interior hashing, Bloom-filter queries, and equality comparisons cover only the header and child-reference words. Leaf hashing covers only the original 64-bit occupancy. The physical allocation size therefore contains one more word than the logical size supplied to deduplication. This preserves the original rule that structural geometry determines sharing and prevents a derived or temporarily uninitialised descriptor from splitting otherwise identical subtrees.

After the geometry DAG has been deduplicated, the memoised pass described in Section 3.6 fills the reserved trailing words before the structure is uploaded for rendering. Since the descriptor is itself a deterministic function of the unique subtree and its level, every spatial reference to a shared node obtains the same result through the existing node address. The construction changes physical node stride and storage cost but does not add a new edge, alter the child mask, or modify traversal's spatial path semantics.

3.10 LOD-Aware Ray Traversal

LOD selection is integrated into the existing hierarchical stack traversal. After the ray descends into an intersected child, the renderer resolves that child's node index and header, evaluates its slab intersection, and reuses the resulting entry parameter for $S_l < d\theta_p\tau$. No second ray–box test is introduced. If LOD is disabled or the projected-size condition fails, traversal proceeds through the child mask exactly as in the baseline path.

The inline descriptor is accessed only after the projected-size test succeeds. This ordering avoids an additional node-tail memory read for nodes that cannot terminate and for all rays when $\tau = 0$. For a candidate stored node, let the normalised ray direction be $\boldsymbol{\omega} = (\omega_x, \omega_y, \omega_z)$. Its dominant axis is

$$a^* = \underset{a\in\{X,Y,Z\}}{\arg\max}\, |\omega_a|. \tag{3.27}$$

Only P_{a^*} is queried. This converts the three axis-aligned construction-time predicates into one view-dependent traversal decision without storing separate values for the two ray signs.

Let k count the number of additional descents already requested by coverage for the current ray. When a node satisfies the screen-space criterion, traversal continues if

$$P_{a^*}(N) = 1 \quad \wedge \quad k < D_{\max}; \tag{3.28}$$

otherwise, the candidate is accepted as the hit node. The current implementation uses $D_{\max} = 5$. Each accepted passability request increments k, so a sequence of sparse nodes can add at most five hierarchy levels beyond the first eligible candidate. Refinement may stop earlier when a descendant is non-passable. The bound prevents highly porous subtrees from eliminating the performance benefit of LOD by forcing unconditional descent to the finest level.

When early termination is accepted at level l, the traversal records three outputs for subsequent stages: the spatial path, the hit level l, and the selected packed descriptor. Partial path coordinates are shifted into the same high-bit-aligned layout used by a full-depth path, leaving the unresolved lower coordinate bits equal to zero. The path therefore denotes the minimum corner of the selected node, and its physical edge length is recovered as $S_l = 2^{L-l}$. The packed word is written to a one-word-per-pixel intermediate buffer because the later shading stage retains the spatial path but no longer has the selected geometry-node index.

Background rays store level zero and an empty descriptor. Full-resolution hits store level L and an empty relief distribution, preserving the original per-voxel result. If termination occurs within one of the two implicit subdivisions encoded inside a 4^3 leaf, no independent stored node descriptor exists at that scale, so the empty distribution selects the box-normal fallback. At all stored-node termination levels, the descriptor corresponds directly to the accepted unique DAG node.

3.11 LOD Appearance Reconstruction

After colour retrieval, the illumination stage reconstructs the surface represented by the selected LOD node. Let $\mathbf{V}$ be the unit direction from the hit point towards the camera, $\mathbf{L}$ the unit light direction, and $\mathbf{n}_b$ the geometric normal of the box face entered by the primary ray. The decoder converts the six quantised fields into $\widehat{r}_d$ values before applying the current view and lighting configuration.

3.11.1 View-Dependent Internal Relief

The stored histogram contains directions from the complete node subtree, including surfaces that face away from the current camera. A decoded bin therefore receives the projected-area weight

$$w_d = \widehat{r}_d \max(0, \mathbf{n}_d \cdot \mathbf{V}) . \tag{3.29}$$

The positive dot product removes back-facing directions and foreshortens an axis-aligned surface according to the current view. This operation is performed at shading time rather than during construction, allowing one node descriptor to serve every camera position. Up to three mutually orthogonal axis directions can contribute for a general view, but their weights depend on the node-specific relief values rather than being selected as an unweighted normal set.

The six-direction representation is not used to infer which coarse boundary the ray crossed. That information comes from a ray–box intersection at the selected node scale. The stored path supplies the node's minimum corner and the hit level supplies S_l; intersecting the camera ray with this box yields the hit position and the unique entry-face normal $\mathbf{n}_b$. Thus, view-dependent relief supplements rather than replaces resolved boundary geometry.

3.11.2 Box-Normal Residual and Diffuse Reconstruction

Let $R = \sum_{d \in \mathcal{D}} w_d$ be the visible relief weight. The part of the projected response not claimed by internal relief is assigned to the intersected box face:

$$w_b = \max(0, 1 - R), \qquad W = R + w_b. \tag{3.30}$$

If visible relief exceeds one projected node face, w_b becomes zero and the relief terms are normalised by their total. Otherwise, the residual preserves the macroscopic box surface. The reconstructed Lambertian response is

$$f_{\text{diff}} = \frac{w_b \max(0, \mathbf{n}_b \cdot \mathbf{L}) + \sum_{d \in \mathcal{D}} w_d \max(0, \mathbf{n}_d \cdot \mathbf{L})}{W}. \tag{3.31}$$

This normalisation keeps the response bounded as the number of visible directions changes. When all relief bins are zero, $R = 0$, $w_b = W = 1$, and Equation 3.31 reduces exactly to ordinary shading with $\mathbf{n}_b$. This includes the completely solid-node case discussed in Section 3.5.2: the raw histogram would mix up to three external face normals, whereas internal-relief separation retains only the face actually hit by the ray.

Figure 3.6 compares the reconstructed appearance at a common LOD scale. Each 4^3 coarse node obtains its own descriptor by aggregating the enclosed 1^3 fine voxels; the relief is not shared merely because the nodes lie at the same level.

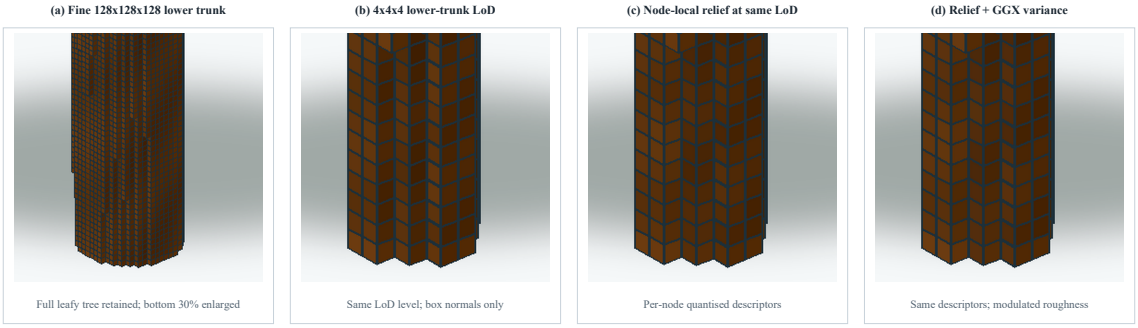

Figure 3.6: Appearance reconstruction for a 128^3 voxel tree, with the bottom 30% square region enlarged to emphasise the trunk. From left to right: fine-voxel reference, 4^3 LOD nodes using representative colour and box normals, node-local internal-relief shading at the same LOD level, and relief shading with GGX roughness modulation. Every coarse descriptor is computed bottom-up from the fine occupancy of that individual node.

3.11.3 Roughness from Directional Variance

A weighted diffuse response restores low-frequency orientation variation, but directly applying the same mixture to a sharp specular model can produce several discrete axis-aligned highlights. The method instead interprets deviation from the resolved box normal as unresolved normal variance. It evaluates one GGX lobe about $\mathbf{n}_b$ and uses the relief distribution only to broaden that lobe.

The second moment of the visible directional distribution about the box-normal axis is

$$s_b = \mathrm{clamp}\left(\frac{w_b \sum_{d\in\mathcal{D}} w_d(\mathbf{n}_d \cdot \mathbf{n}_b)^2}{W}, 0, 1\right). \tag{3.32}$$

The residual box component contributes unit alignment. Relief parallel to the box normal also contributes one, whereas orthogonal directions contribute zero. Thus, $1 - s_b$ is a compact measure of the normal spread unresolved by the selected LOD node.

Let α_0 be the base GGX slope parameter and λ the variance scale. The effective squared parameter supplied to the GGX distribution and Smith visibility terms is

$$\alpha_{\mathrm{eff}}^2 = \mathrm{clamp}\big(\alpha_0^2 + \lambda(1 - s_b), \alpha_0^2, 1\big). \tag{3.33}$$

The current renderer uses a base perceptual roughness of 0.5, giving $\alpha_0 = 0.5^2 = 0.25$ and $\alpha_0^2 = 0.0625$, with $\lambda = 1$. The final specular response is evaluated once as

$$f_{\text{spec}} = f_{\text{GGX}}\left(\mathbf{n}_b, \mathbf{V}, \mathbf{L}; \alpha_{\text{eff}}^2\right). \tag{3.34}$$

Keeping $\mathbf{n}_b$ as the lobe axis preserves the macroscopic orientation established by the actual ray–box hit. A solid node or other zero-relief boundary has $s_b = 1$ and retains the original GGX response. A node containing orthogonal unresolved surfaces obtains a broader highlight, reducing abrupt high-frequency variation without creating multiple competing specular lobes.

3.12 Method Summary

The proposed method extends HashDAG rendering with a screen-space termination rule and node-local information for reconstructing the geometry that early termination removes. Projected node size first identifies an LOD candidate. Complete six-direction coverage then supplies three thresholded passability decisions, allowing sparse candidates to request at most five additional descent levels. Representative colours are obtained at the accepted geometry scale from the associated multi-resolution colour representation.

For appearance reconstruction, exact 4^3 leaf statistics are aggregated bottom-up over unique DAG nodes. Separating boundary area from total exposed area produces internal relief that excludes the box face already resolved by the ray. The six quantised relief values, three coverage-derived passability bits, and one reserved bit form a single inline 32-bit descriptor using the 555544 directional allocation. The trailing word is excluded from hashing and equality, preserving structural deduplication.

At shading time, the current view filters the six relief directions, while the actual ray-entry face retains the box-normal residual. The weighted distribution reconstructs diffuse response and provides a directional-variance term that broadens one GGX lobe about the correct box normal. The complete pipeline therefore combines traversal reduction, sparse-region refinement, representative colour, and stable coarse shading without reconstructing the terminated subtree or introducing a second primary-ray pass.

Chapter 4

Evaluation

4.1 Experimental Setup

The evaluation examines three aspects of the proposed method: its effect on runtime rendering performance, its ability to preserve image quality under increasingly aggressive LOD thresholds, and the construction and storage overhead introduced by the inline prefilter. A separate controlled experiment analyses the effect of directional variance on the GGX response. Unless stated otherwise, all measurements use the same scene, executable configuration, and rendering resolution.

4.1.1 Hardware and Software Environment

All experiments were performed on a laptop equipped with an NVIDIA GeForce RTX 3060 Laptop GPU with 6 GB of graphics memory, an Intel Core i7-11800H processor, and approximately 16 GB of system memory. The operating system was Windows 10 Home 22H2, build 19045, with NVIDIA driver 596.36. The project was compiled for 64-bit Release using the MSVC v143 toolset and CUDA 13.1. Rendering used a fixed output resolution of 1280×960 pixels.

The test scene was Epic Citadel voxelised at depth $L = 17$, corresponding to a finest-grid resolution of 2^{17} cells along each axis. The same stored scene data were used by every runtime configuration. The coverage threshold and maximum additional descent were fixed to the values used throughout Chapter 3, namely $T_{\mathrm{cov}} = 0.35$ and $D_{\max} = 5$. Compressed colour-leaf retrieval used one deterministic central sample, $K = 1$.

4.1.2 Runtime Configurations

The performance experiment compares four configurations. *Original HashDAG* disables LOD with $\tau = 0$ and follows the original full-resolution traversal. *Box LOD* enables screen-space termination at $\tau = 0.6$ but disables the directional prefilter, isolating the benefit and error introduced by replacing a subtree with its box. *Relief LOD* uses the same threshold and enables representative colour, internal-relief shading, and roughness modulation, while leaving coverage-aware additional descent disabled. *Complete method* additionally enables the coverage-derived passability test. The last two configurations therefore differ only in whether sparse candidate nodes can request bounded refinement.

Setting	Original HashDAG	Box LOD	Relief LOD	Complete method
τ	0.0	0.6	0.6	0.6
Relief	Off	Off	On	On
Roughness variance	–	–	On	On
Coverage refinement	Off	Off	Off	On

Table 4.1: Configurations used for the runtime performance comparison. A dash denotes a component that is not applicable because prefiltered shading is disabled.

For the image-quality experiment, the full-resolution image with $\tau = 0$ is used as the reference. Box-normal, relief, and coverage-aware relief rendering are evaluated at $\tau \in \{0.5, 1.0, 1.5\}$. Both relief configurations enable the same GGX roughness modulation, so their difference isolates coverage-aware refinement. Lighting, shadows, fog, camera pose, output resolution, and all remaining sampling settings are held constant across the ten captures.

4.1.3 Performance Measurement Protocol

Runtime measurements use the 998-frame camera path stored in `epiccitadel_move.csv`. Each frame records GPU rendering time using CUDA events. The reported value is the sum of primary-path traversal, colour resolution, and the configured illumination stage; window presentation, user-interface rendering, and vertical synchronisation are excluded. Shadows are disabled in all four performance configurations so that differences in secondary-ray traversal do not obscure the cost of the primary LOD method.

The replay is executed twice for each independently compiled configuration. The first pass warms the GPU and scene data, and only written to the measurement file in the second time. Curves are aligned by replay frame number. Aggregate results report the arithmetic mean frame time, the ratio between the Original HashDAG mean and each method mean, and the largest aligned per-frame ratio.

4.1.4 Image-Quality and Construction Protocols

Image-quality captures use a fixed camera pose and are saved directly from the renderer's RGBA8 output texture without the user interface. Metrics are evaluated on RGB values in stored sRGB space. Peak signal-to-noise ratio (PSNR) measures per-pixel reconstruction error, structural similarity (SSIM) measures local luminance, contrast, and structural agreement, and LPIPS measures distance in AlexNet feature space. Higher PSNR and SSIM and lower LPIPS indicate closer agreement with the full-resolution reference. In addition to full-frame metrics, a 240×240 region centred at pixel $(285, 752)$, using a top-left image origin, is enlarged for qualitative inspection.

Construction and memory costs are measured with two Release builds that differ only in whether inline prefilter storage and construction are enabled. Coverage-aware traversal is disabled in both builds because it does not change the stored descriptor size. Source BasicDAG file loading is excluded. The timed interval includes HashDAG construction, optional prefilter construction, colour-hierarchy construction, and GPU upload. Logical geometry storage, allocated geometry pages, colour storage, and their combined logical size are recorded after construction. Each construction configuration was measured once; these timings are therefore reported as observed costs rather than estimates with statistical variation.

4.2 Runtime Performance

4.2.1 Per-Frame Behaviour

Figure 4.1 plots the raw GPU time for every aligned replay frame. The cost of the Original HashDAG varies substantially with the camera path. It begins near 17 ms, falls below 10 ms around frame 150, and reaches its most expensive interval between approximately frames 350 and 600, where several peaks exceed 20 ms. This variation shows why a single stationary view would not adequately represent traversal performance: the number and depth of intersected regions change as the camera moves through the scene.

All three LOD configurations reduce the large peaks. Box LOD and Relief LOD remain close throughout the replay and reduce the highest-cost interval to approximately 15 ms. The similarity of these two curves indicates that decoding the six relief bins and reconstructing their diffuse and specular response adds little cost relative to the traversal saved by early termination. The Complete method is consistently above the other two LOD curves in the expensive interval, reaching approximately 18 ms at its largest peak. This is expected because low-coverage candidates may descend by up to five additional levels before termination.

After approximately frame 620, the four curves converge into a range of roughly 7–10 ms. In this part of the replay, the full-resolution traversal is already comparatively inexpensive, so there is less work for LOD to remove. The benefit is therefore concentrated in views for which the original method encounters a large number of costly fine-level queries rather than being a constant offset across the complete camera path.

4.2.2 Aggregate Performance

Table 4.2 summarises the aligned measurements. Box LOD obtains the lowest mean frame time, reducing the Original HashDAG result from 11.694 ms to 9.314 ms, a mean

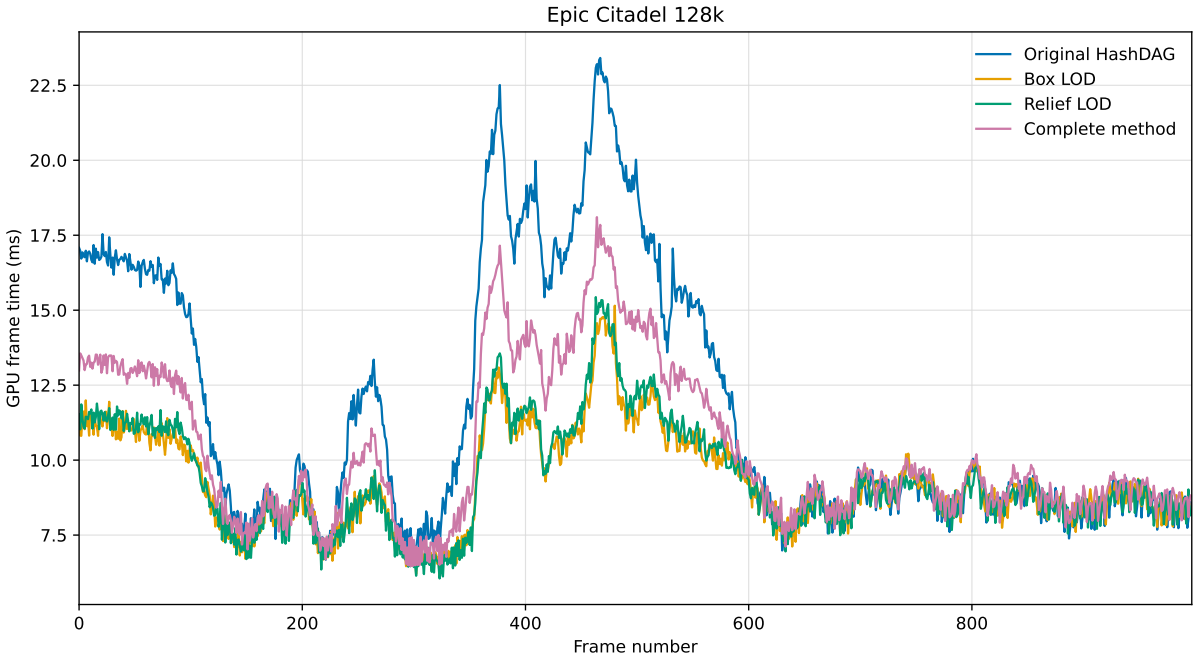

Figure 4.1: Raw GPU frame time over the 998-frame Epic Citadel replay. Each configuration was measured on the second of two aligned replay passes. The value contains the CUDA-event timings of the rendering stages and excludes presentation and user-interface work.

speedup of 1.26×. Relief LOD records 9.378 ms and 1.25×, only 0.064 ms slower on average than Box LOD. The node-local appearance reconstruction therefore retains nearly all of the performance benefit of screen-space termination in this experiment.

Configuration	Mean time (ms)	Mean speedup	Maximum frame speedup
Original HashDAG	11.694	1.00×	1.00×
Box LOD	9.314	1.26×	1.80×
Relief LOD	9.378	1.25×	1.72×
Complete method	10.363	1.13×	1.41×

Table 4.2: Aggregate GPU frame-time results for the 998 aligned replay frames. Mean speedup is the ratio of mean frame times, while maximum speedup is the largest Original-to-method ratio at any corresponding frame.

Coverage-aware descent increases the Complete method mean to 10.363 ms. Relative to Relief LOD, this is an additional 0.985 ms, or approximately 10.5%, but the result remains 1.13× faster than full-resolution rendering. The maximum aligned per-frame speedups follow the same ordering: 1.80× for Box LOD, 1.72× for Relief LOD, and 1.41× for the Complete method. These maxima describe individual camera positions and should not be interpreted as sustained acceleration; the mean values capture the overall replay more reliably.

The performance ordering exposes the cost of progressively restoring information

removed by coarse termination. Box LOD provides the upper bound on traversal acceleration for the selected threshold. Relief reconstruction preserves substantially more appearance information at negligible additional mean cost, whereas coverage-aware refinement deliberately returns some of the saved traversal work to prevent sparse nodes from behaving as opaque boxes. Whether that additional cost is justified is evaluated through the image-quality results in the following section.

4.3 Image Quality Evaluation

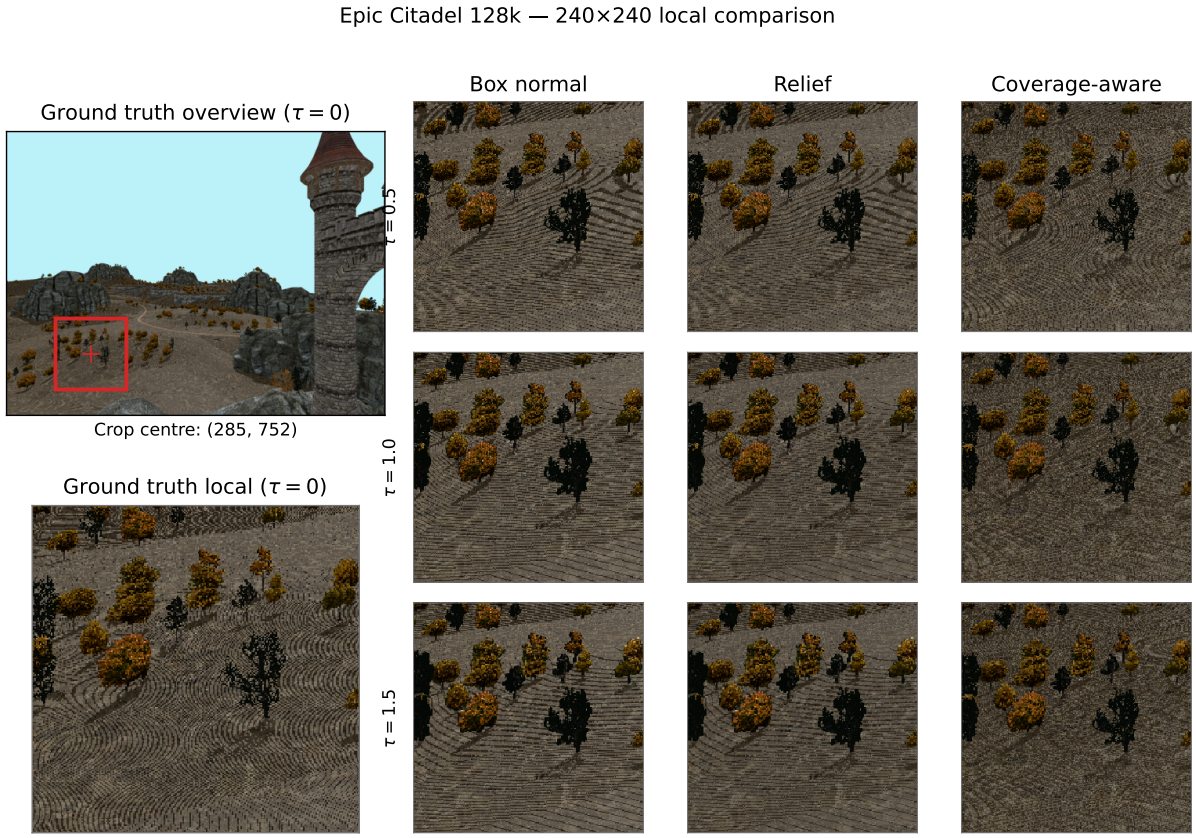

Figure 4.2: Visual comparison against the full-resolution reference. The rows use increasingly permissive LOD thresholds, while the columns isolate box-normal shading, relief reconstruction, and coverage-aware refinement. Each inset shows the same 240×240 crop centred at pixel $(285, 752)$.

4.3.1 Visual Comparison

Figure 4.2 enlarges the fixed crop used for qualitative inspection. Box-normal shading suppresses directional variation within terminated nodes and makes thin or partially occupied regions appear broader and more continuous than in the reference. Relief reconstruction restores variation in surface response, but it cannot recover a fine-scale silhouette once traversal has already terminated. Its shading therefore visibly improves

while its structural similarity remains close to that of the box-normal result, as quantified in the following subsection.

The coverage-aware result retains more gaps and boundary detail in sparse vegetation and other partially occupied regions by requesting bounded additional descent. Its advantage becomes most apparent at $\tau = 1.5$, where the other methods accept the coarsest terminations. Some difference from the reference remains because refinement is limited to $D_{\max} = 5$ and still ends at an LOD representation rather than necessarily reaching the finest voxels. The visual ordering therefore agrees with the objective measurements without implying exact reconstruction.

4.3.2 Objective Image Metrics

Figure 4.3 and Table 4.3 compare each LOD configuration with the full-resolution reference. PSNR measures pixel-wise reconstruction fidelity, SSIM compares local luminance, contrast, and structure [13], and LPIPS compares deep features associated with perceptual similarity [15]. The three metrics consistently show increasing error as τ rises, because a larger screen-space threshold permits traversal to terminate at coarser nodes.

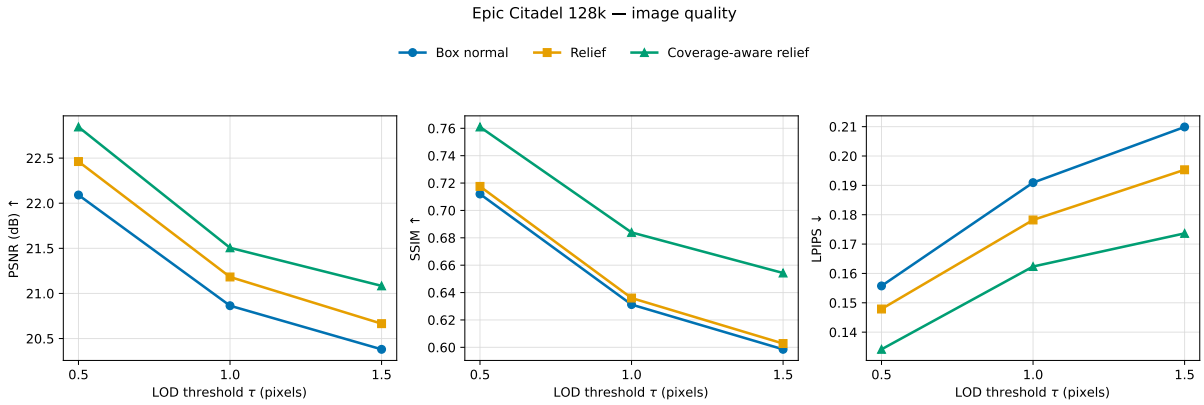

Figure 4.3: Full-frame image-quality metrics relative to the $\tau = 0$ reference. Higher values are better for PSNR and SSIM, whereas lower values are better for LPIPS.

Replacing box-normal shading with relief reconstruction increases PSNR by 0.371, 0.319, and 0.284 dB at $\tau = 0.5$, 1.0, and 1.5, respectively. In contrast, the corresponding SSIM increases are only 0.0055, 0.0048, and 0.0042. This difference is consistent with the role of the relief descriptor: it corrects local tonal and directional shading errors, which affect pixel-wise error, but does not alter the coarse termination footprint or restore geometry omitted by the selected node. The local structures compared by SSIM consequently remain very similar to those of Box LOD. LPIPS nevertheless decreases at every threshold, by 0.008, 0.013, and 0.015, showing that relief reconstruction also improves perceptual agreement.

Method	τ	PSNR (dB) ↑	SSIM ↑	LPIPS ↓
Box normal	0.5	22.091	0.712	0.156
Relief	0.5	22.462	0.718	0.148
Coverage-aware relief	0.5	**22.845**	**0.761**	**0.134**
Box normal	1.0	20.864	0.631	0.191
Relief	1.0	21.182	0.636	0.178
Coverage-aware relief	1.0	**21.506**	**0.684**	**0.162**
Box normal	1.5	20.381	0.599	0.210
Relief	1.5	20.665	0.603	0.195
Coverage-aware relief	1.5	**21.084**	**0.654**	**0.174**

Table 4.3: Objective image quality relative to the full-resolution reference. The best result at each threshold is shown in bold.

Coverage-aware refinement produces a larger structural improvement. Relative to Relief LOD, it raises SSIM by 0.043, 0.048, and 0.052 across the three thresholds and lowers LPIPS by 0.014, 0.016, and 0.022. Its PSNR is also highest in every group. These results support the intended separation of responsibilities: relief improves the appearance assigned to a terminated node, whereas the coverage test prevents selected sparse nodes from replacing visibly discontinuous fine geometry with a single opaque coarse footprint.

4.4 Specular and Roughness Analysis

4.4.1 Controlled Response Comparison

The full-scene comparison combines geometric, colour, visibility, and shading errors. A controlled experiment is therefore used to isolate the response of the relief and roughness terms. Figure 4.4 selects an exterior leaf region from the same voxel data used by the appearance comparison in Figure 3.6. Fine 1^3 voxels are aggregated into one 4^3 node at a fixed LOD level, and each incident direction is aimed at the selected region. Only the outward-facing hemisphere is evaluated so that the response represents surfaces that can be observed from outside the node rather than hidden back-facing geometry.

Panel (a) evaluates the fine voxels and acts as the directional reference. Its multiple changes in response arise because different incident directions reach different exposed fine-voxel faces. Panel (b) replaces the region with its coarse bounding box and uses the first-hit box normal. This retains the resolved macroscopic boundary but loses the unresolved distribution of orientations. Panel (c) adds the view-weighted internal relief derived from the selected node, and panel (d) further converts its directional second moment into the effective GGX roughness defined in Chapter 3. The lower plots show the absolute response change from panel (a) for the three coarse alternatives.

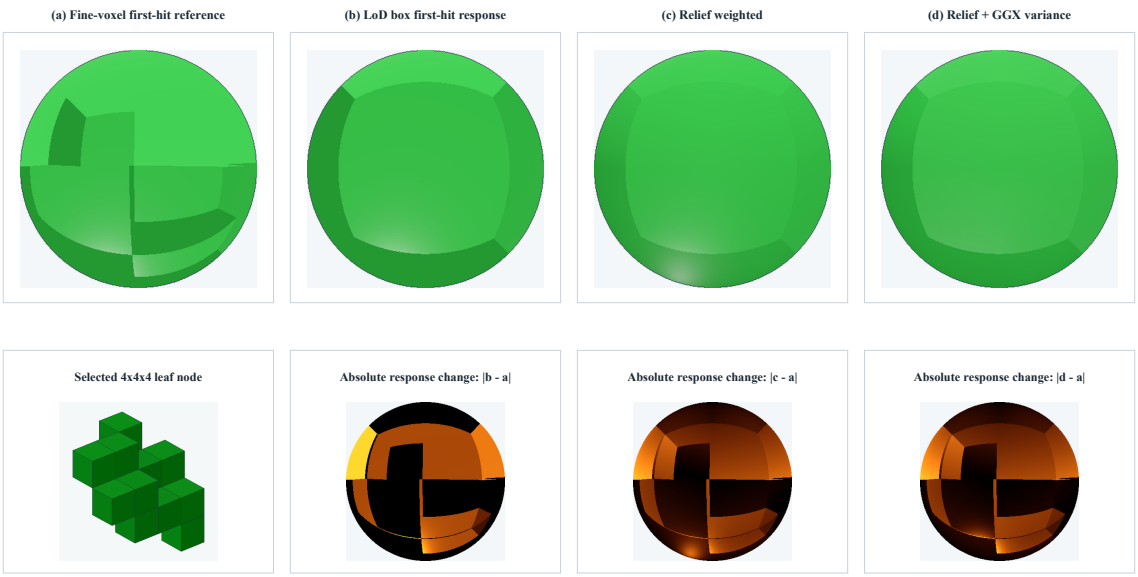

Figure 4.4: Controlled directional response of one exterior 4^3 node aggregated from 1^3 fine voxels at a common LOD level. (a) Fine-voxel reference; (b) coarse first-hit box normal; (c) node-local, view-weighted relief without roughness modulation; and (d) the same relief response with directional-variance GGX modulation. The lower row reports the absolute response change from the fine reference for (b)–(d). The descriptor and variance are computed from this node's own fine occupancy rather than prescribed for the LOD level.

4.4.2 Interpretation

The box-normal curve is governed by a single resolved face and consequently cannot reproduce the sequence of fine-surface responses. Relief weighting reintroduces the proportions of visible axis-aligned internal surfaces, reducing the dependence on one coarse normal. Applying those directions as separate sharp specular lobes would, however, replace one discontinuity with several axis-aligned highlights. The proposed modulation instead retains the actual entry-face normal as the centre of one GGX lobe and uses directional variance only to broaden it.

This distinction is important for completely solid nodes. A raw six-axis surface histogram would contain contributions associated with multiple external faces, even though a particular ray intersects only one of them. Mixing those normals would create an erroneous highlight. Internal-relief separation removes the resolved boundary contribution during construction, so a solid node has zero relief and its response falls back to the correct first-hit box normal. Roughness modulation is introduced only when the node contains unresolved internal orientation variance. Panel (d) therefore illustrates a smoothing of the coarse specular response without changing the resolved boundary orientation or requiring additional rays. The experiment explains the mechanism of the shading model; the scene-level quality metrics in Section 4.3 remain the evidence for its effect in the complete renderer.

4.5 Construction and Memory Cost

4.5.1 Construction Time

Table 4.4 reports the observed construction time with and without the inline prefilter. Enabling the prefilter increases the geometry phase from 37.347 to 100.686 s. This phase includes HashDAG construction, bottom-up descriptor construction, and geometry upload, so the 63.338 s increase captures the complete preprocessing path rather than only the arithmetic used to pack the descriptor. Total measured construction rises from 43.613 to 109.996 s, an increase of 152.21%.

Configuration	Geometry and upload (s)	Colour hierarchy (s)	Total (s)
Without prefilter	37.347	6.265	43.613
Inline prefilter	100.686	9.311	109.996
Observed increase	169.59%	48.61%	152.21%

Table 4.4: Observed Release construction times for Epic Citadel. BasicDAG file loading is excluded. Each configuration was measured once.

The colour representation is unchanged by the geometry descriptor, although its separately timed phase is 3.045 s longer in the prefilter run. Because only one run was recorded for each build, this difference cannot be separated from run-to-run variation and is not attributed structurally to the prefilter. The geometry increase is the meaningful implementation cost: exact leaf statistics, sibling-interface correction, and memoised bottom-up aggregation add offline work before rendering. This cost is paid when the static scene is constructed and does not recur per frame.

4.5.2 Storage Overhead

Measurement	Without prefilter	Inline prefilter	Increase
Logical geometry (MB)	983.363	1172.881	19.27%
Allocated geometry (MB)	1513.869	1776.656	17.36%
Colour (MB)	3366.945	3366.945	0.00%
Logical total (MB)	4350.309	4539.826	4.36%

Table 4.5: Storage after HashDAG construction. Logical total combines logical geometry and colour storage; allocated geometry additionally reflects page-allocation granularity.

The logical and allocated storage results are given in Table 4.5. The trailing 32-bit word raises logical geometry storage from 983.363 to 1172.881 MB, an increase of 189.517 MB or 19.27%. Geometry-page allocation increases by 262.787 MB, or 17.36%, because the larger node records also alter page packing and allocation granularity. These allocated

figures describe reserved geometry capacity and are therefore higher than the logical bytes occupied by node records.

Colour storage remains exactly 3366.945 MB because the descriptor is appended only to geometry nodes. Consequently, the 19.27% logical geometry increase becomes a 4.36% increase in total logical scene storage. This result reflects the intended compact integration: all six quantised relief values, three passability decisions, and the reserved bit add one word per stored node without duplicating the colour hierarchy or introducing a separate node-indexed lookup structure.

Chapter 5

Discussion and Limitations

The evaluation demonstrates that screen-space termination can reduce traversal cost while node-local relief and coverage-aware refinement recover part of the appearance lost at coarse LODs. Nevertheless, the current descriptor remains an approximation of fine geometry rather than a visibility-complete representation. Its compact size is beneficial for traversal and storage, but it necessarily discards information that can be important in sparse, layered, or heavily occluded regions.

The first limitation concerns directional coverage. The current implementation estimates coverage along axis (a) as

$$C_a = \frac{A_{-a} + A_{+a}}{2S^2}. \tag{5.1}$$

This quantity is derived from exposed surface area. It therefore approximates how many independent solid segments occur along the projected voxel columns, rather than directly measuring how many columns are occupied. Several leaves positioned behind one another can increase (C_a), even when they overlap in the same projected region and do not increase screen-space coverage. A more accurate way is projecting the node and performing a logical OR along every voxel column.

With logical OR, a column contributes once whether it contains one occupied voxel or five overlapping occupied segments. Computing and aggregating such projected occupancy masks would better represent occlusion, but would require additional construction data and potentially more storage than the three passability bits retained by the present design.

The second limitation is that relief weighting cannot fully reconstruct the true fine-surface response. Boundary subtraction prevents the external faces of a completely solid node from being mixed with the normal of the box face actually hit by the ray. It does not, however, determine whether the remaining internal surfaces are visible from outside the node. For example, a node with a closed outer shell and an entirely enclosed cavity is consequently assigned internal relief, although the cavity surfaces cannot contribute to any external view. Similar errors occur with several layers of geometry along the same viewing direction. Correctly resolving these cases would require visibility-aware directional statistics rather than exposed-area aggregation alone. The

present method should therefore be understood as a low-cost reconstruction of orientation distribution, not an exact prefilter of fine-scale visibility and reflectance.

The GGX BRDF used by the renderer is consistent with common real-time game-rendering practice, and the directional-variance term reduces some abrupt coarse-level highlight changes. However, applying a conventional microfacet model directly to discrete voxel geometry can still produce aliasing, moiré patterns, and unstable distant highlights. These effects become more visible as many fine voxel faces collapse into a small number of pixels. In addition, the BasicDAG currently stores only a limited subset of the material information normally used by a production BRDF. Parameters such as spatially varying reflectance, normal maps, and other material attributes are not generally baked into the voxel hierarchy because of the trade-off between voxel resolution and graphics-memory consumption. The resulting GGX response is therefore constrained by incomplete material data and cannot match the richness of a fully parameterised surface-rendering pipeline.

Finally, the current LOD data are designed for static construction and are not yet maintained robustly under interactive editing. Relief, coverage, representative colour, and roughness variance are subtree-derived quantities, so a geometry or material edit can invalidate descriptors along affected ancestor paths. DAG sharing further complicates this process because one unique node may be referenced by multiple spatial instances. A complete editable solution would require incremental descriptor reconstruction, correct copy-on-write behaviour, and synchronisation with the existing hash-based deduplication process. The same limitation applies to export: the current pipeline does not yet provide a complete, versioned representation that guarantees the inline descriptors and multi-resolution appearance data remain consistent when a modified scene is serialised and loaded elsewhere. Incremental maintenance and export integration are therefore left as future work.

Chapter 6

Conclusion and Future Work

This work introduced an appearance-aware LOD extension for HashDAG rendering. Screen-space termination is integrated directly into hierarchical ray traversal, allowing sub-pixel regions to be represented by internal DAG nodes instead of requiring descent to the finest voxels. To reduce the resulting loss of appearance, the method combines multi-resolution colour retrieval with node-local internal relief, directional roughness variance, and coverage-derived passability. These quantities are constructed bottom-up and stored as one inline 32-bit descriptor per geometry node, retaining the compact and shared structure of the original DAG.

The evaluation on Epic Citadel demonstrates the resulting performance–quality trade-off. At the evaluated runtime threshold, Box LOD and Relief LOD reduce mean frame time by factors of 1.26× and 1.25×, respectively, while the complete coverage-aware method remains 1.13× faster than full-resolution traversal. Relief reconstruction improves PSNR and LPIPS over box-normal shading with little additional runtime cost, whereas coverage-aware descent provides the largest improvement in structural similarity and visual preservation of sparse regions. This additional quality is obtained at a higher traversal cost than Relief LOD alone. The inline descriptor increases total logical scene storage by 4.36%, although the present offline construction implementation also introduces a substantial one-time build cost.

Future work should first replace the exposed-area coverage approximation with projected occupancy information. Performing a logical OR along each voxel column, or constructing a compact hierarchical representation of these projected masks, would avoid counting several depth-overlapping segments as additional screen-space coverage. Such a representation could improve refinement decisions in layered vegetation and similarly sparse geometry without relying on a single global threshold to compensate for overestimation.

Relief construction should likewise become visibility-aware. Boundary subtraction correctly removes the resolved outer faces of a solid node, but does not distinguish externally visible internal structure from closed cavities or fully occluded layers. Depth-aware directional statistics, transmittance estimates, or compact visibility moments could suppress surfaces that cannot contribute to an exterior view. A richer directional representation could also reduce the axis-aligned approximation, provided that its memory and decoding costs remain compatible with the traversal savings.

The shading model could be extended by storing and prefiltering more complete material information in the voxel hierarchy. Spatially varying reflectance, material roughness, normal variation, and other BRDF parameters would permit a closer match to production rendering pipelines. Their integration should be accompanied by explicit distant-geometry filtering or temporal stabilisation to address voxel aliasing, moiré patterns, and unstable specular highlights rather than relying on GGX roughness modulation alone.

Finally, the descriptor must be integrated with the editable and serialised forms of the HashDAG. Geometry and material edits should incrementally invalidate and reconstruct affected node statistics while preserving copy-on-write semantics and hash-based deduplication. Exported scenes should include a versioned descriptor layout and sufficient metadata to validate or rebuild multi-resolution appearance data after loading. Addressing these issues would extend the current static rendering method into a complete LOD system for editable, portable voxel scenes.

References

[1] S. Bako, P. Sen, and A. Kaplanyan. Deep appearance prefiltering. *ACM Transactions on Graphics*, 42(2):23:1–23:23, 2023.

[2] V. Careil, M. Billeter, and E. Eisemann. Interactively modifying compressed sparse voxel representations. *Computer Graphics Forum*, 39(2):111–119, 2020.

[3] C. Crassin, F. Neyret, S. Lefebvre, and E. Eisemann. Gigavoxels: Ray-guided streaming for efficient and detailed voxel rendering. In *Proceedings of the ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games*, pages 15–22, 2009.

[4] Crytek. Sponza. Crytek CRYENGINE, 2010. 3D scene model.

[5] B. Dado, T. R. Kol, P. Bauszat, J.-M. Thiery, and E. Eisemann. Geometry and attribute compression for voxel scenes. *Computer Graphics Forum*, 35(2):397–407, 2016.

[6] C. Han, B. Sun, R. Ramamoorthi, and E. Grinspun. Frequency domain normal map filtering. *ACM Transactions on Graphics*, 26(3):28, 2007.

[7] E. Heitz, J. Dupuy, C. Crassin, and C. Dachsbacher. The SGGX microflake distribution. *ACM Transactions on Graphics*, 34(4):48:1–48:11, 2015.

[8] E. Heitz and F. Neyret. Representing appearance and pre-filtering subpixel data in sparse voxel octrees. In *Eurographics/ACM SIGGRAPH Symposium on High Performance Graphics*, pages 125–134, 2012.

[9] V. Kämpe, E. Sintorn, and U. Assarsson. High resolution sparse voxel DAGs. *ACM Transactions on Graphics*, 32(4):101:1–101:13, 2013.

[10] S. Laine and T. Karras. Efficient sparse voxel octrees. In *Proceedings of the ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games*, pages 55–63, 2010.

[11] D. Meagher. Geometric modeling using octree encoding. *Computer Graphics and Image Processing*, 19(2):129–147, 1982.

[12] M. Toksvig. Mipmapping normal maps. *Journal of Graphics Tools*, 10(3):65–71, 2005.

[13] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.

[14] P. Weier, T. Zirr, A. Kaplanyan, L.-Q. Yan, and P. Slusallek. Neural prefiltering for correlation-aware levels of detail. *ACM Transactions on Graphics*, 42(4):78:1–78:16, 2023.

[15] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 586–595, 2018.

[16] Y. Zhou, T. Huang, R. Ramamoorthi, P. Sen, and L.-Q. Yan. Appearance-preserving scene aggregation for level-of-detail rendering. *ACM Transactions on Graphics*, 44(1):8:1–8:23, 2025.